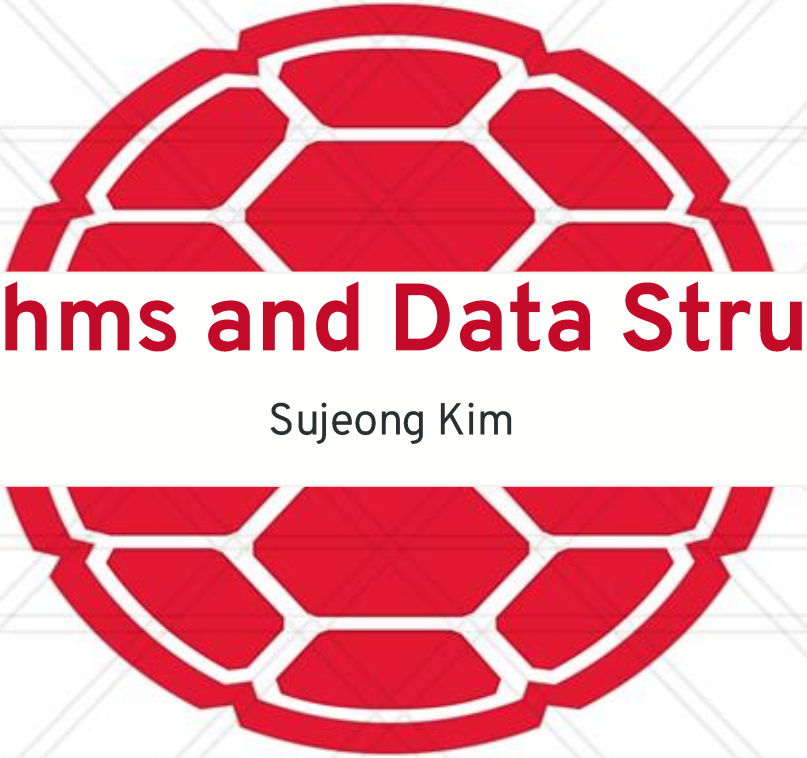


Algorithms and Data Structures

Sujeong Kim

UNIVERSITY OF
MARYLAND

Lecture 15-16 Graphs

Slide Credit: These slides are adapted from materials by Prof. Manocha, with modifications and additions by Sujeong Kim.

Graphs

- A collection of vertices or nodes, connected by a collection of edges.

- Applicable to many applications where there is some “connection” or “relationship” or “interaction” between pairs of objects
  - social network analysis
  - network communication & transportation
  - VLSI design & logic circuit design
  - surface meshes in CAD/CAM & GIS
  - path planning for autonomous agents
  - precedence constraints in scheduling

Basic Definitions

- Directed Graph (or digraph) G = (V, E) consists of a finite set V, called vertices or nodes, and E, a set of ordered pairs, called edges of G.
- E is a binary relation on V.
- Self-loops are allowed.
- Multiple edges (between the same pair of vertices in the same direction) are not allowed, though (v, w) and (w, v) are distinct edges.

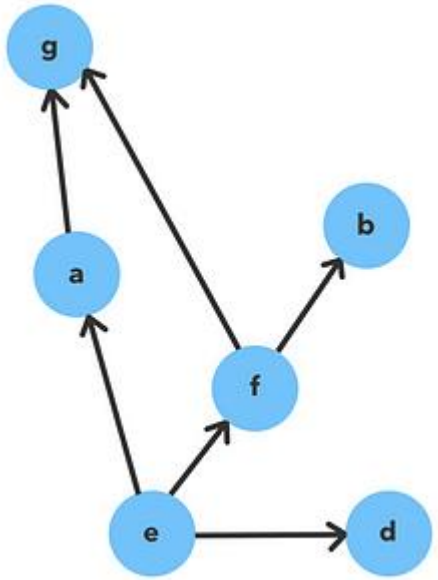


Image: medium.com

Basic Definitions

- Undirected Graph (or graph) G = (V, E) consists of a finite set V of vertices, and a set E of unordered pairs of distinct vertices, called edges of G. No self-loops are allowed.

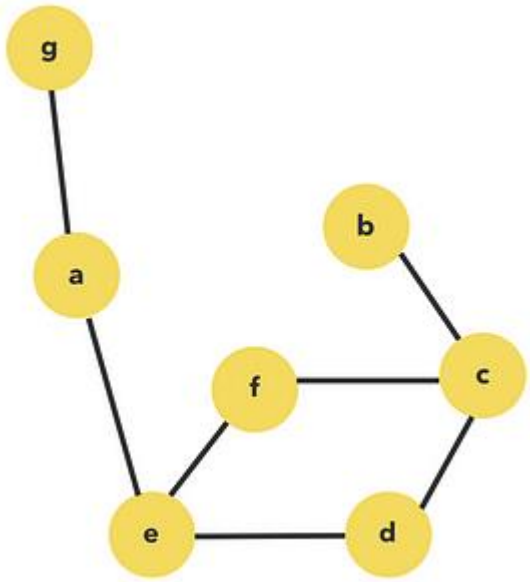


Image: medium.com

Examples of Digraphs & Graphs

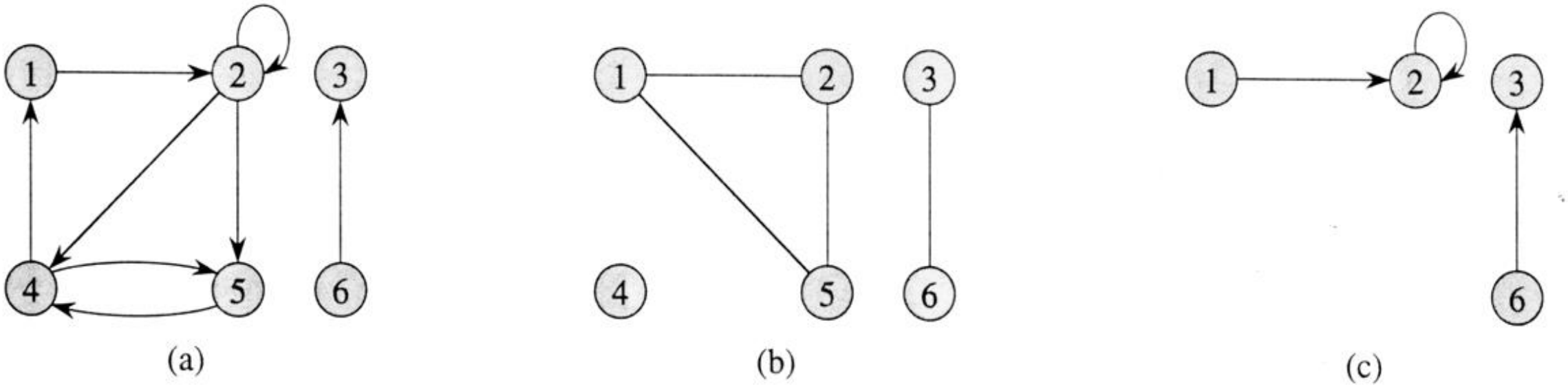


Figure 5.2 Directed and undirected graphs. **(a)** A directed graph $G = (V, E)$, where $V = \{1, 2, 3, 4, 5, 6\}$ and $E = \{(1, 2), (2, 2), (2, 4), (2, 5), (4, 1), (4, 5), (5, 4), (6, 3)\}$. The edge $(2, 2)$ is a self-loop. **(b)** An undirected graph $G = (V, E)$, where $V = \{1, 2, 3, 4, 5, 6\}$ and $E = \{(1, 2), (1, 5), (2, 5), (3, 6)\}$. The vertex 4 is isolated. **(c)** The subgraph of the graph in part (a) induced by the vertex set $\{1, 2, 3, 6\}$.

Definitions

- Vertex w is adjacent to vertex v if there is an edge (v,w).

- In an undirected graph:
  - Edge e = (u,v) : This edge connects vertices u and v.
  - Endpoints: u and v are the endpoints of e.
  - Incidence: The edge e is said to be incident on both u and v.
    * Incident in graph theory describes the connection between vertices and edges.

- In a directed graph (digraph):
  - Edge e = (u,v) : This edge has a direction, going from vertex u to vertex v.
  - Origin: Vertex u is where the edge starts (leaves).
  - Destination: Vertex v is where the edge ends (enters).
  - Incidence: The edge e leaves u and enters v.

Definitions

- A digraph or graph is weighted if its edges are labeled with numeric values.

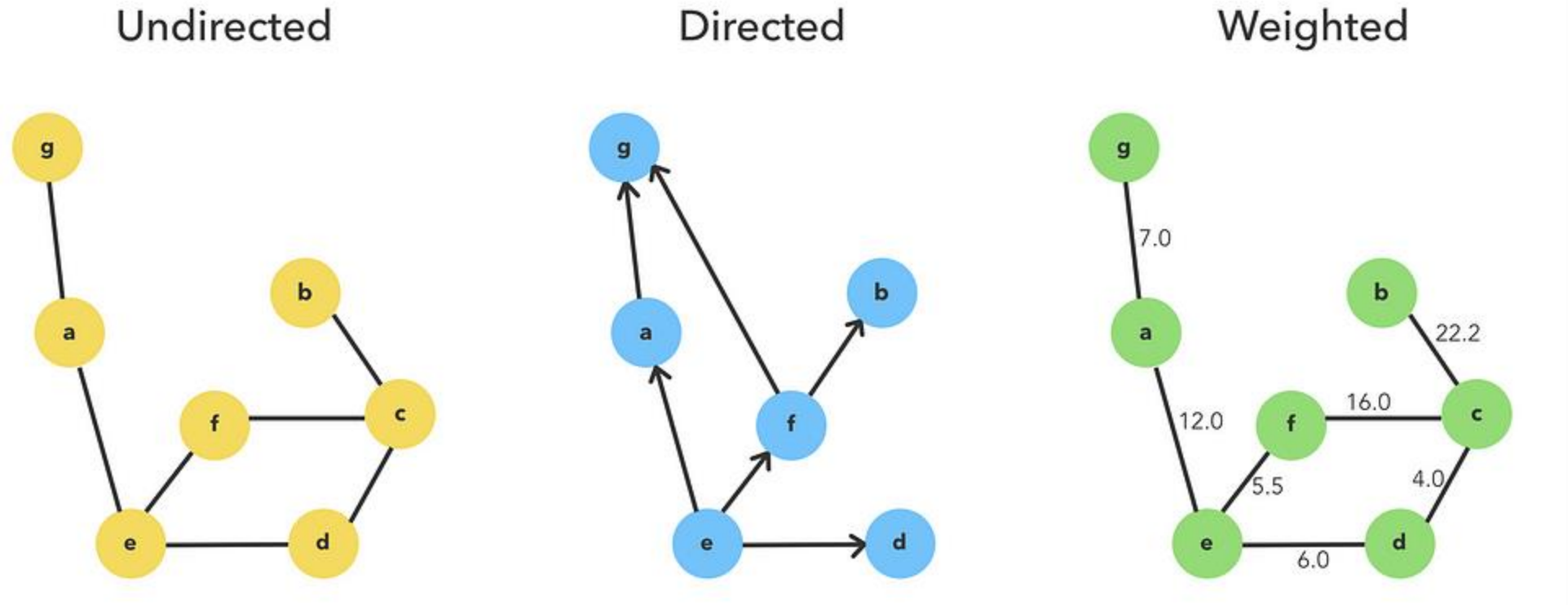


Image: medium.com

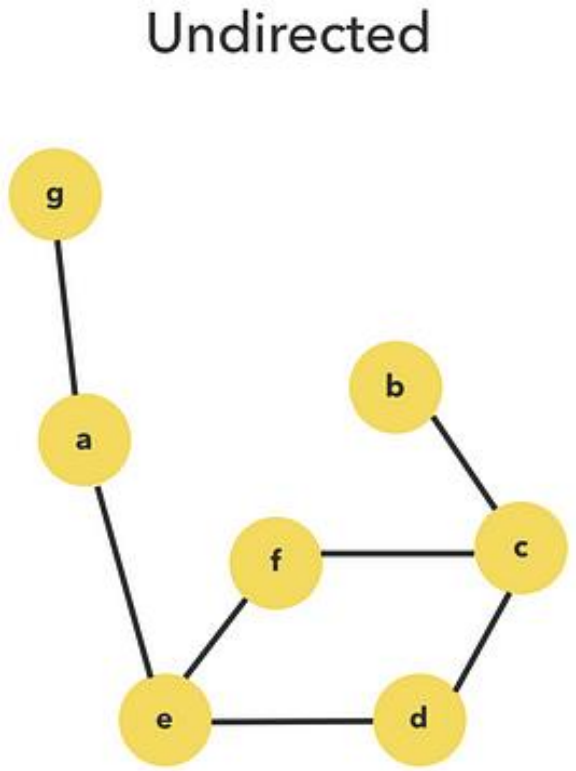
Definitions - Degree

- Undirected Graph
  - Degree of a vertex v : number of edges incident to v.

- Degree sum formula

$$\sum_{v \in V} \deg(v) = 2e$$

The sum of the degrees of all vertices in the graph equals twice the number of edges. This is because each edge contributes to the degree of two vertices.

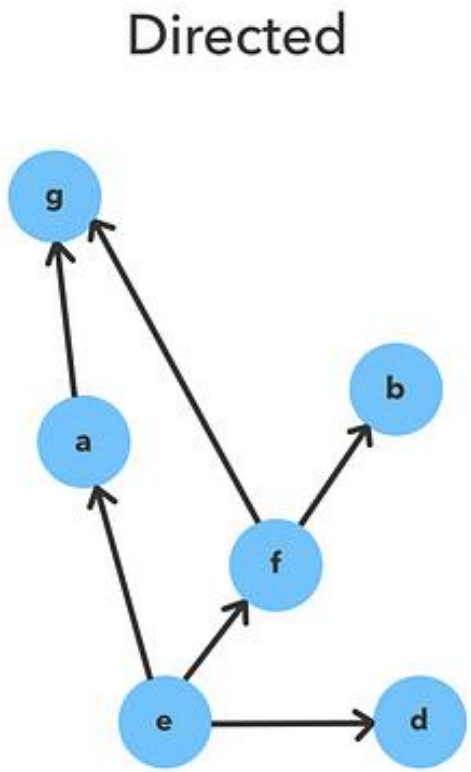


Definitions - Degree

- Directed Graph (Digraph)
  - Out-degree of v: number of edges coming out of v
  - In-degree of v: number of edges coming into v

- Sum of the in-degrees of all vertices equals the sum of the out-degrees of all vertices

$$\sum_{v \in V} \text{indeg}(v) = \sum_{v \in V} \text{outdeg}(v) = e$$



Number of Possible Edges

- 4 vertices (n=4): A, B, C, D
- All possible choice of starting vertex u and destination vertex v (u,v):

AA, AB, AC, AD,
BA, BB, BC, BD,
CA, CB, CC, CD,
DA, DB, DC, DD

- Number of possible edges (e)?
  - Directed graph (no self-loops)
  - Directed graph (with self-loops)
  - Undirected graph (no self-loops)

Combinatorial Facts – Undirected Graph

- Number of edges e

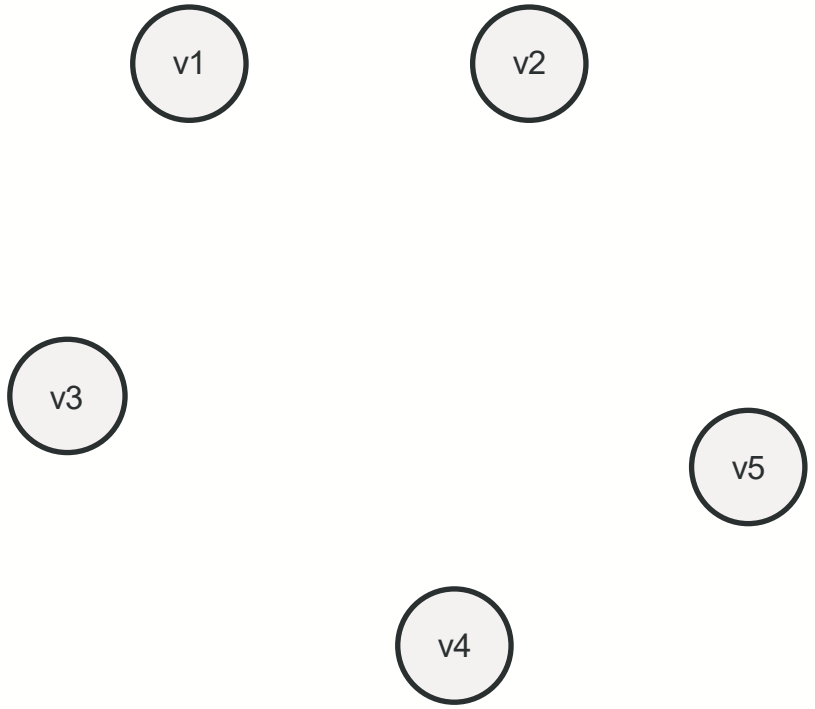
$$0 \leq e \leq C(n,2) = \binom{n}{2} = \frac{n(n-1)}{2} \in O(n^2)$$

  - $C(n,r)$: number of ways to choose **r** elements from a set of **n** elements without considering the order

$$C(n,r) = \binom{n}{r} = \frac{n!}{r!\,(n-r)!}$$

  - The number of edges in an undirected graph is at most $\frac{n(n-1)}{2}$, where n is the number of vertices.

Maximum Number of Edges in an Undirected Graph

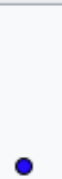
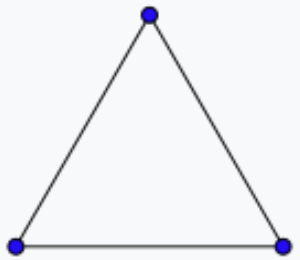
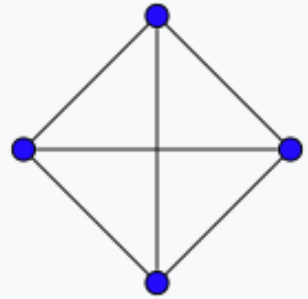
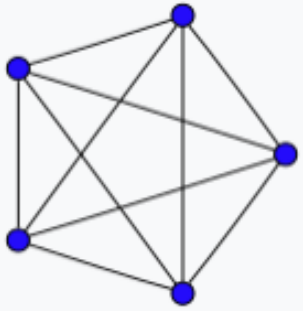
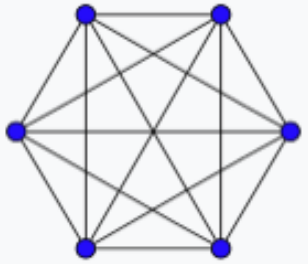
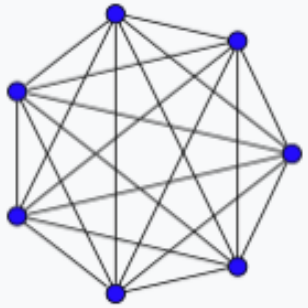
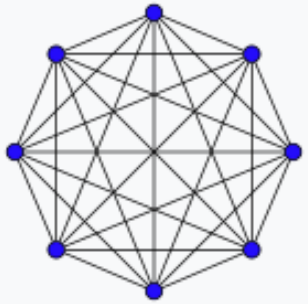
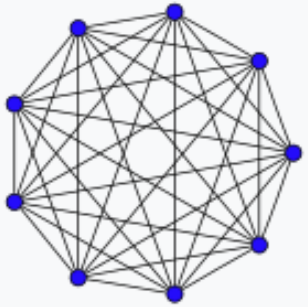
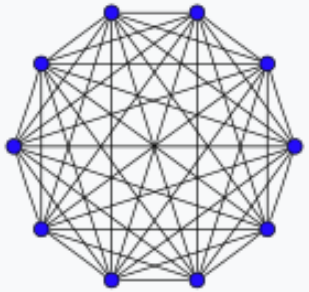
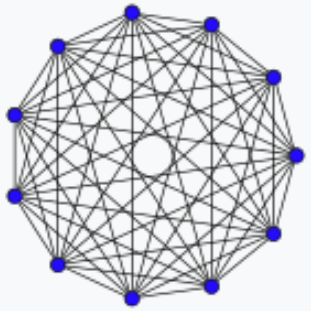
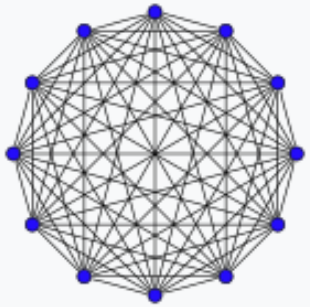


# vertices	# edges (max)	edges
1		
2		
3		
4		
5		
…		
n		

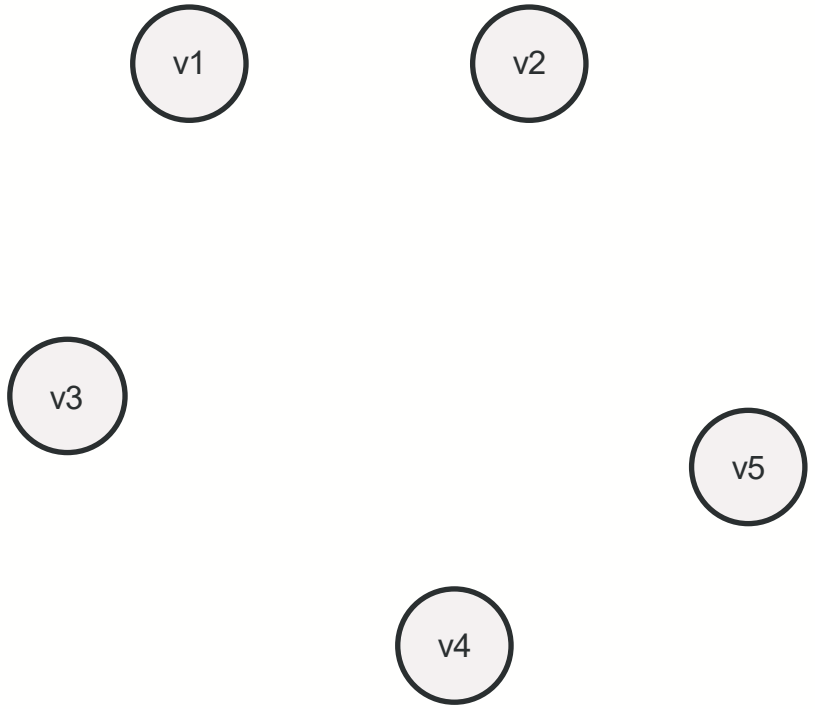
$$\sum_{i=1}^{n}(i-1) = \frac{n(n-1)}{2}$$

Complete Graphs (=fully connected) on n vertices

$$0 \leq e \leq \frac{n(n-1)}{2}$$

K_1: 0	K_2: 1	K_3: 3	K_4: 6
			
K_5: 10	K_6: 15	K_7: 21	K_8: 28
			
K_9: 36	K_{10}: 45	K_{11}: 55	K_{12}: 66
			

Number of Edges in a Directed Graph (Self-loop allowed)



# vertices	# edges (max)	edges
1		
2		
3		
4		
5		
…		
n		

$$\sum_{i=1}^{n}(1+2(i-1)) = \sum_{i=1}^{n} 1 + \sum_{i=1}^{n} 2(i-1) = n + 2\left(\frac{n^2-n}{2}\right) = n + n^2 - n = n^2$$

Combinatorial Facts – Directed Graph

- Number of edges e

$$0 \leq e \leq n^2$$

  - The number of edges e in a digraph is at most n^2. Each pair of vertices can have two directed edges (one in each direction).
  - This counts all possible directed edges between any pair of vertices and self-loops.

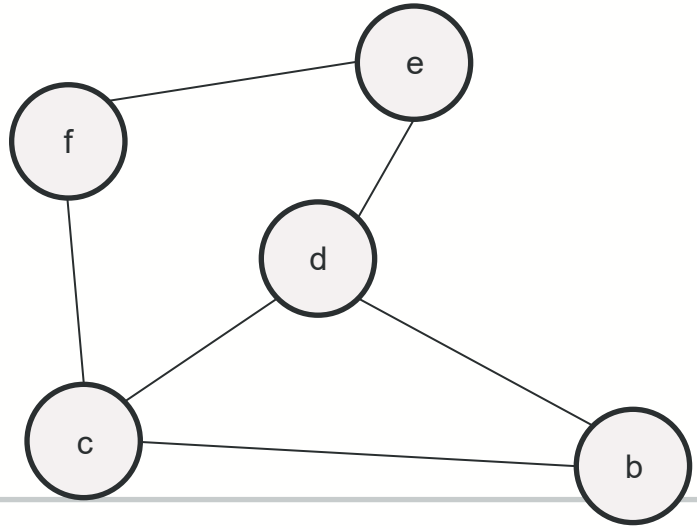
Sparsity and Density

- Sparse Graph
  - A sparse graph has relatively few edges compared to the number of vertices.
    - E.g., Trees, which have $n - 1$ edges for n vertices, are an example of sparse graphs. Every node except the root has exactly one incoming edge.
  - A graph is considered sparse if $e \in O(n)$.

- Dense Graph
  - A graph is considered dense if the number of edges e is close to the maximum possible number of edges: $e \in O(n^2)$.
    - E.g., complete graphs

Definitions - Path

- Path: a sequence of vertices $< v_0, \ldots, v_k >$ s.t. (v_{i-1}, v_i) is an edge for $i = 1 \text{ to } k$, in a digraph.
- In an undirected graph, edges do not have a direction, so the path can be traversed in either direction.
- The length of the path is the number of edges, k.
- w is reachable from u if there is a path from u to w.
- A path is simple if all vertices are distinct.
  - Each vertex is visited exactly once
  - E.g., d-b is simple and the shortest
  - E.g., d-e-f-c-b is simple (but not the shortest)
  - E.g., c-d-e-f-c-b is not simple



Definitions - Cycle

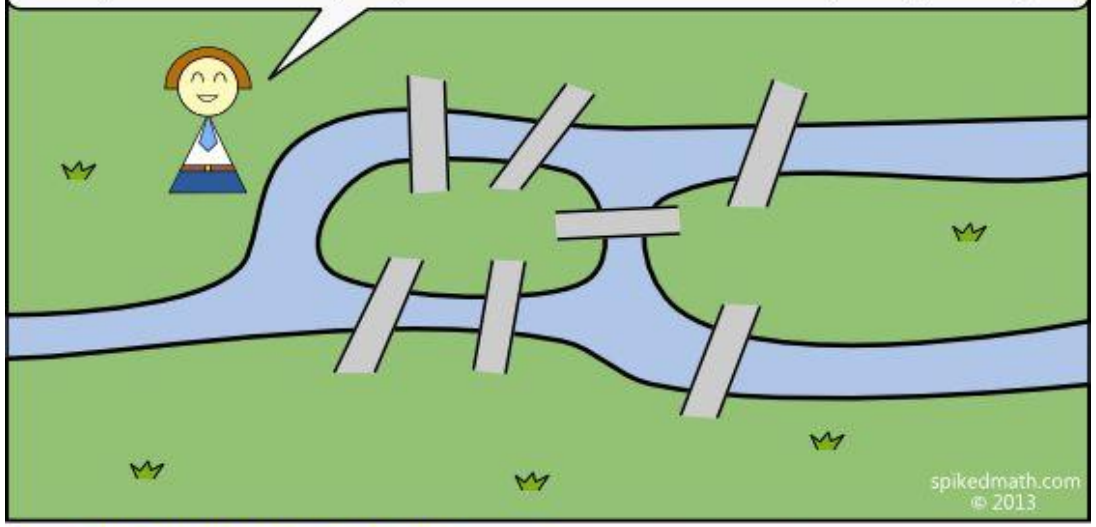
- Cycle
  - A path containing at least 1 edge and for which $v_0 = v_k$

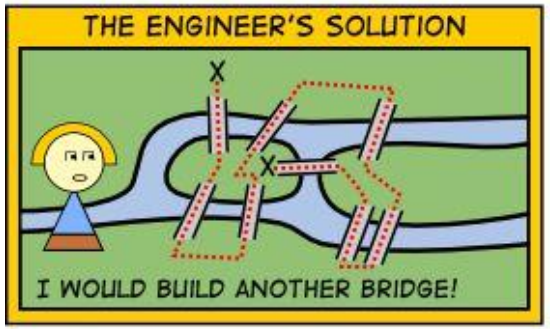
- A cycle is *simple* if, in addition, all intermediate vertices are distinct.
  - E.g., A→B→C→A is a simple cycle in a directed graph.
  - E.g., A-B-C-A is a simple cycle in an undirected graph.

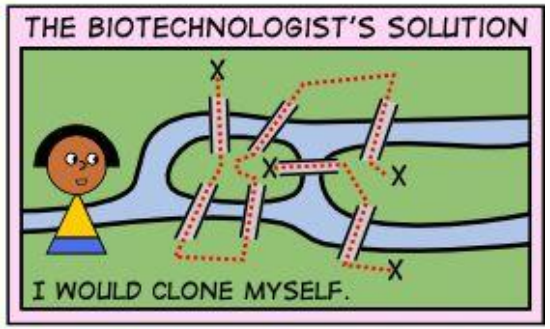
- For undirected *graphs*, a *simple cycle* must visit $\geq$ 3 distinct vertices.
  - E.g., A-B-A is a cycle but not a simple cycle

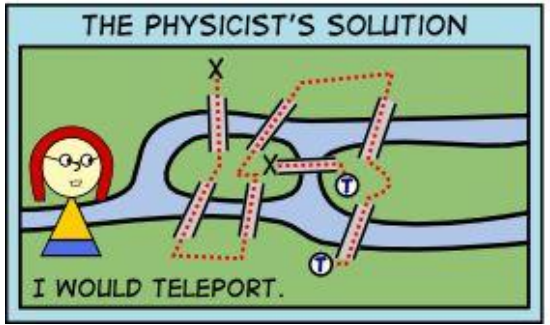
The Seven Bridges of Königsberg

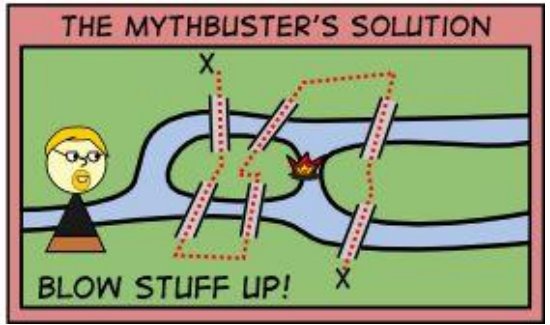
Below is the city of Königsberg with four land masses and seven bridges connecting the various land masses. Can you find a walk through the city of Königsberg that crosses each bridge exactly once? You may start at any land mass you wish but may only travel between land masses by using a bridge.











source

History on Cycles/Paths

- *Eulerian cycle*
  - A cycle that visits every edge of a graph exactly once and returns to the starting vertex. It may revisit vertices.
  - Complexity: Can be solved in polynomial time - $O(E)$, E is at most $O(n^2)$,

Eulerian Cycles/Paths

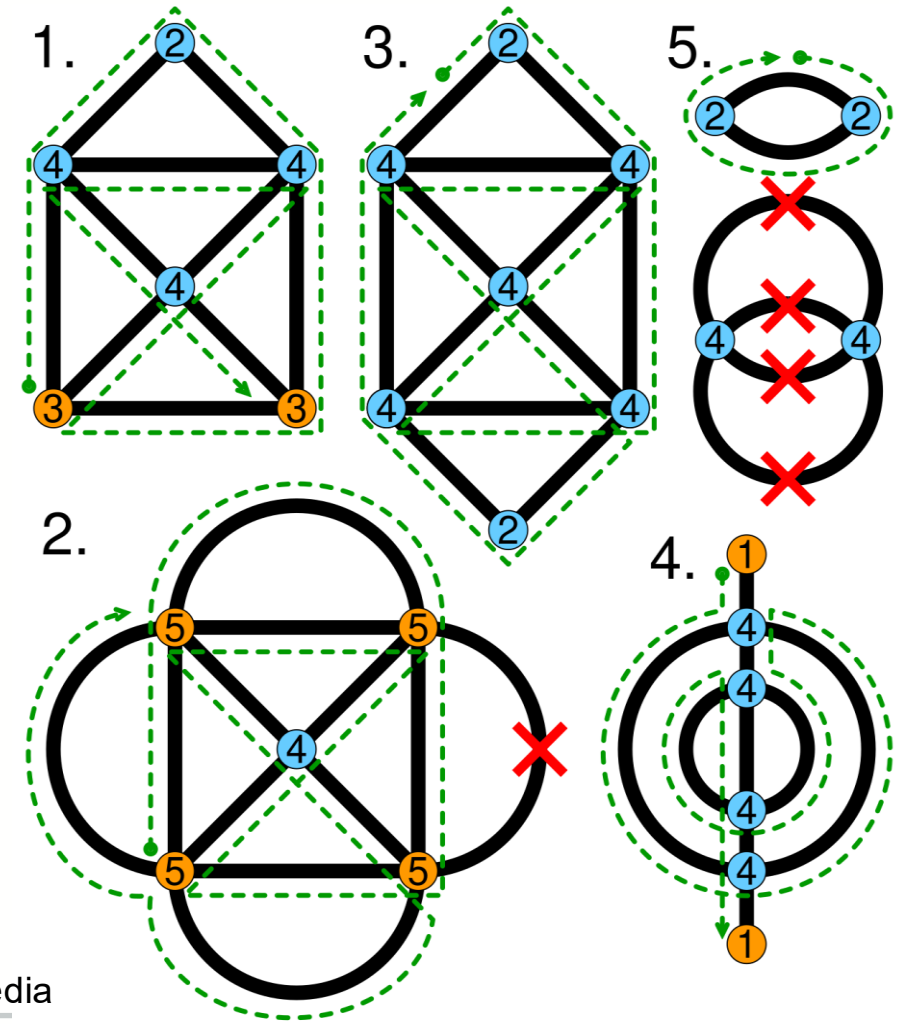


image: wikipedia

- **Eulerian Cycle**: If every vertex has an even degree, the diagram has an Eulerian cycle, meaning you can start and end at the same vertex.
- **Eulerian Path**: If exactly two vertices have an odd degree, the diagram has an Eulerian path, meaning you can start at one odd-degree vertex and end at the other.

Eulerian Cycles/Paths

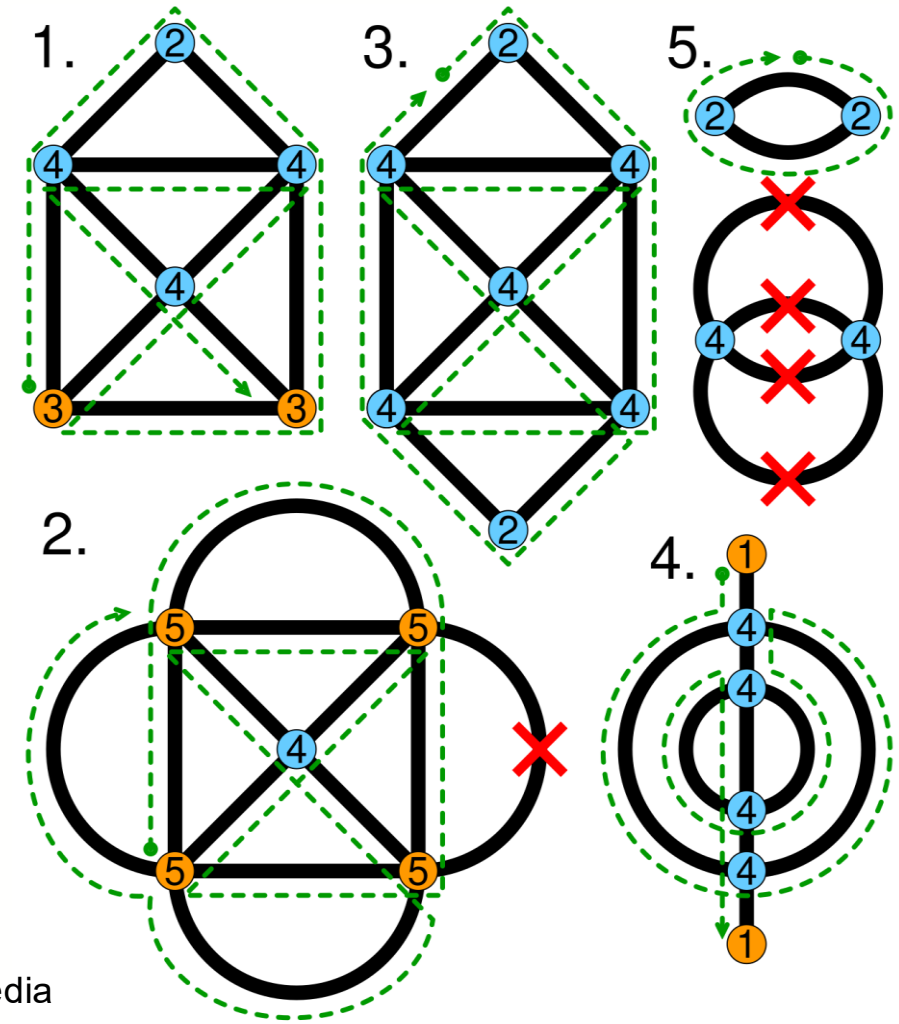
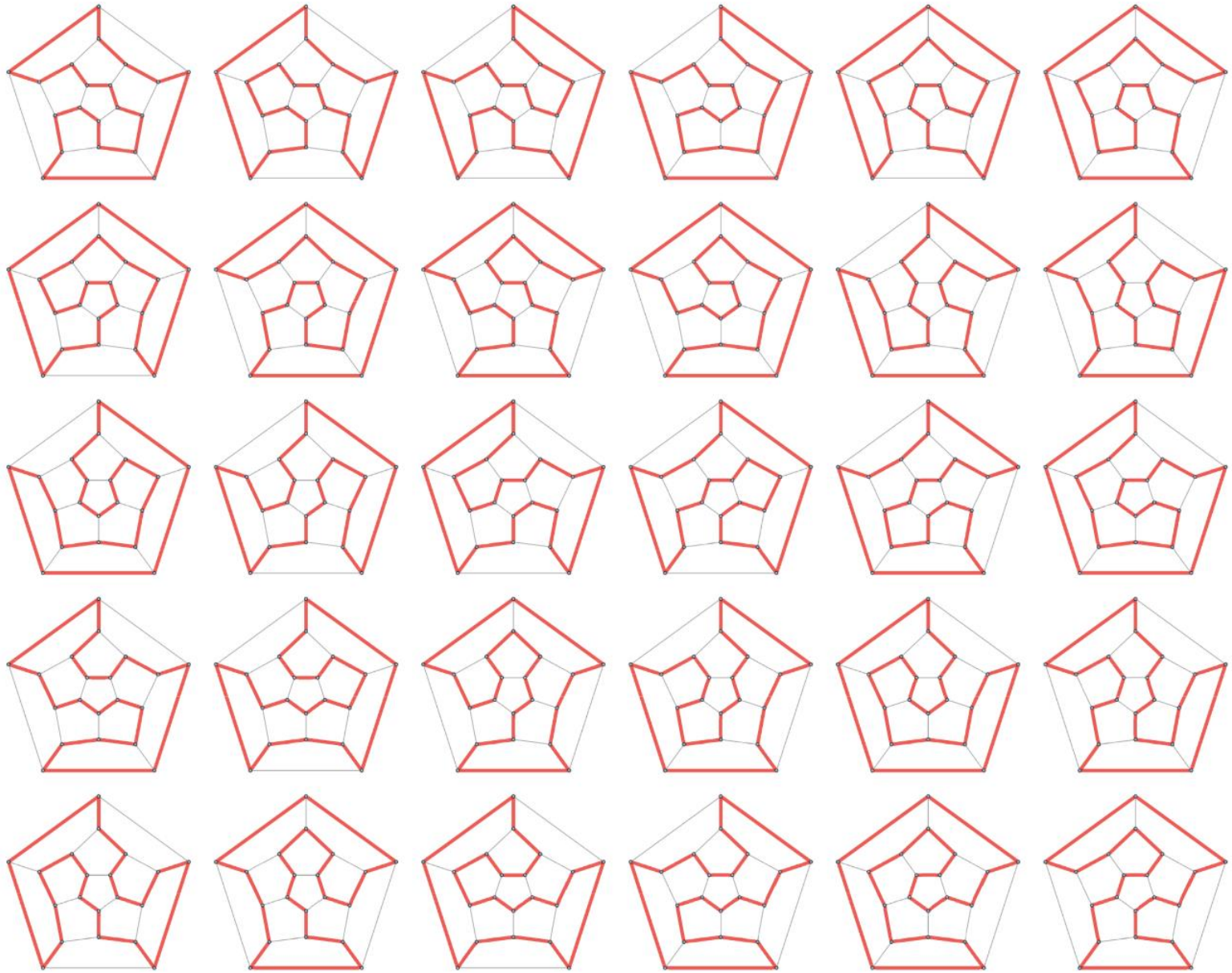


image: wikipedia

- Real world examples
  - Garbage collection: The route planned for the garbage truck should cover every street exactly once.
  - Electrical circuit design: Ensures that every connection is made, and every node is visited without duplicating connections

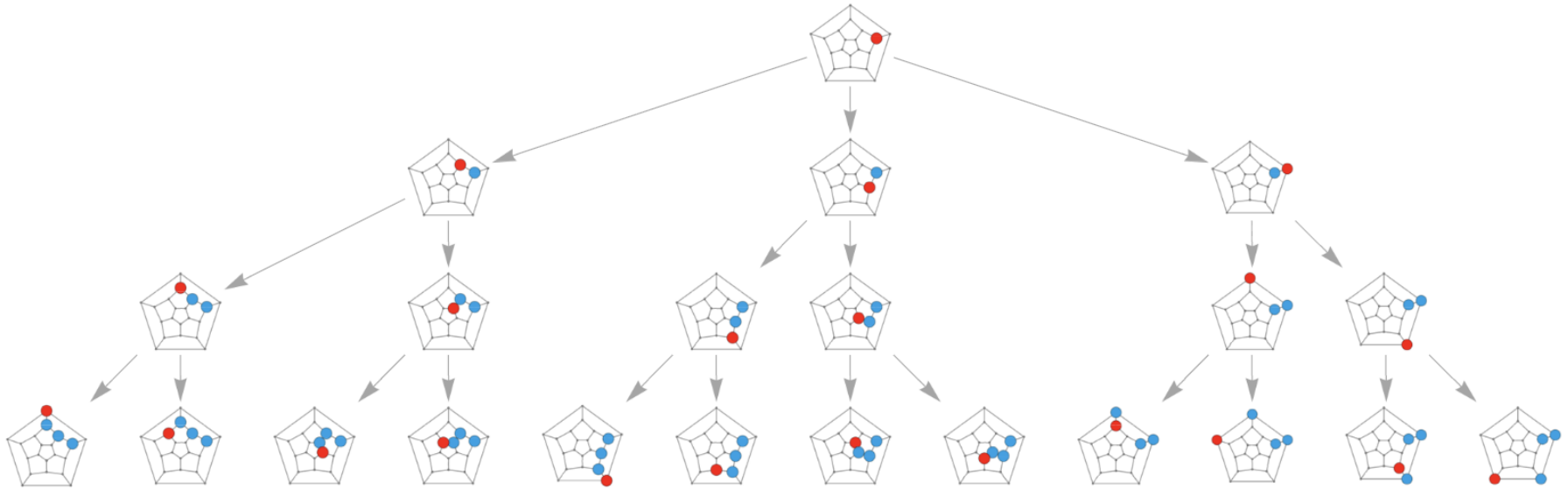
Using Eulerian trails to solve puzzles involving drawing a shape with a continuous stroke



https://mathworld.wolfram.com/IcosianGame.html

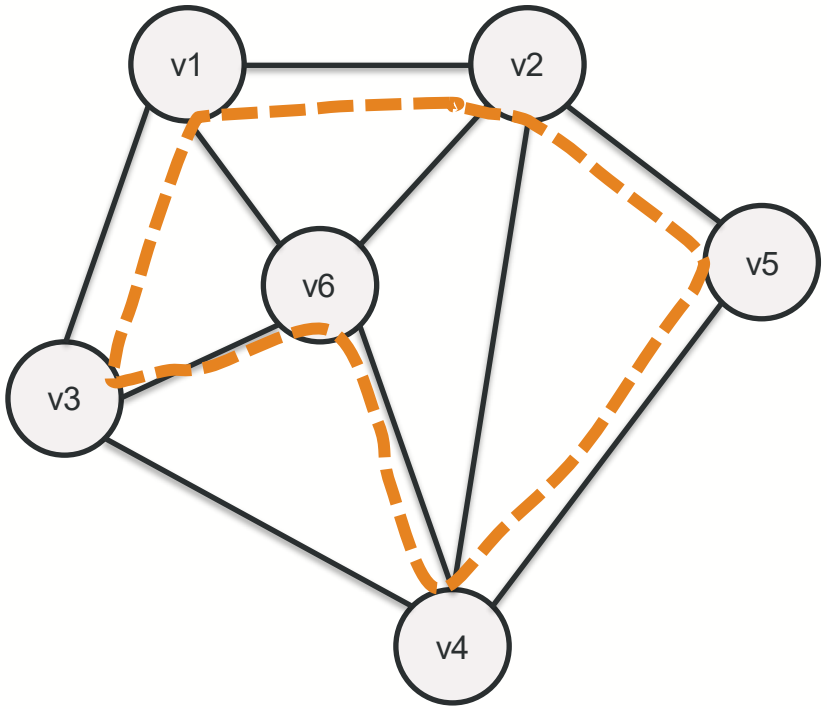
History on Cycles/Paths

- *Hamiltonian cycle*
  - A cycle that visits every vertex exactly once and returns to the starting vertex. It may revisit edges.
  - Complexity: NP-complete problem – computationally challenging



https://mathworld.wolfram.com/IcosianGame.html

Hamiltonian Cycle



Real-world examples

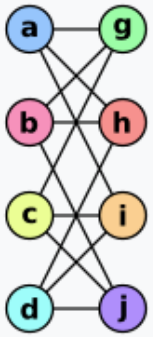
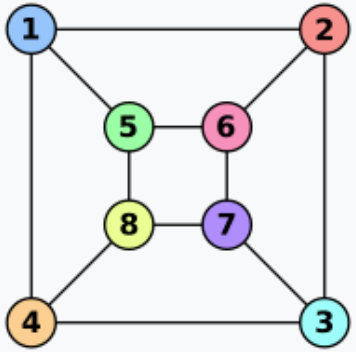
- Traveling salesperson problem (TSP): The route planned visits each city (vertex) exactly once, ensuring that the salesperson covers every city efficiently.

Definitions (Connectivity)

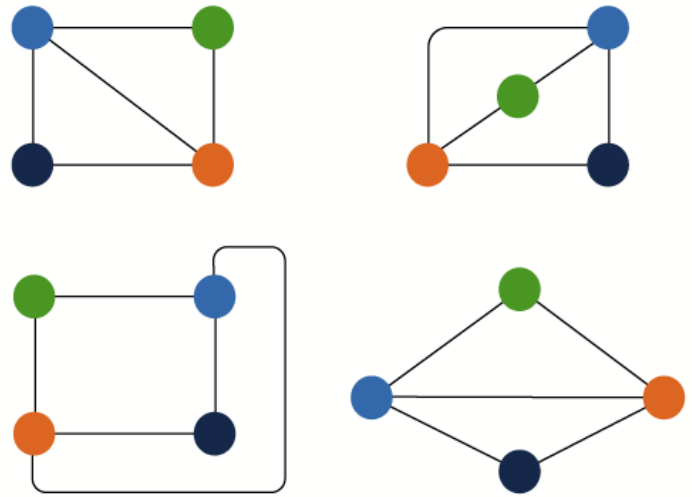
- Acyclic: if a graph contains no simple cycles
  - Example: Trees
- Connected: if every vertex of a graph can reach every other vertex
  - i.e., there exists a path between every pair of vertices
- Connected Components: In an undirect graph, equivalence classes of vertices under “is reachable from” relation
  - Example: In a graph with multiple disconnected subgraphs, each subgraph is a connected component.
- Strongly connected: In a directed graph, if every pair of vertices can reach each other via directed paths
- Fully connected: every pair of vertices is directly connected by an edge
  - e.g., complete graph (undirected) or complete digraph (directed)

Definitions (Connectivity)-contd.

- Isomorphic Graphs: Two graphs $G = (V, E)$ and $G' = (V', E')$ are *isomorphic* if there exists a bijection (one-to-one correspondence) $f : V \rightarrow V'$ s.t. $v, u \in E$ iff $(f(v), f(u)) \in E'$.

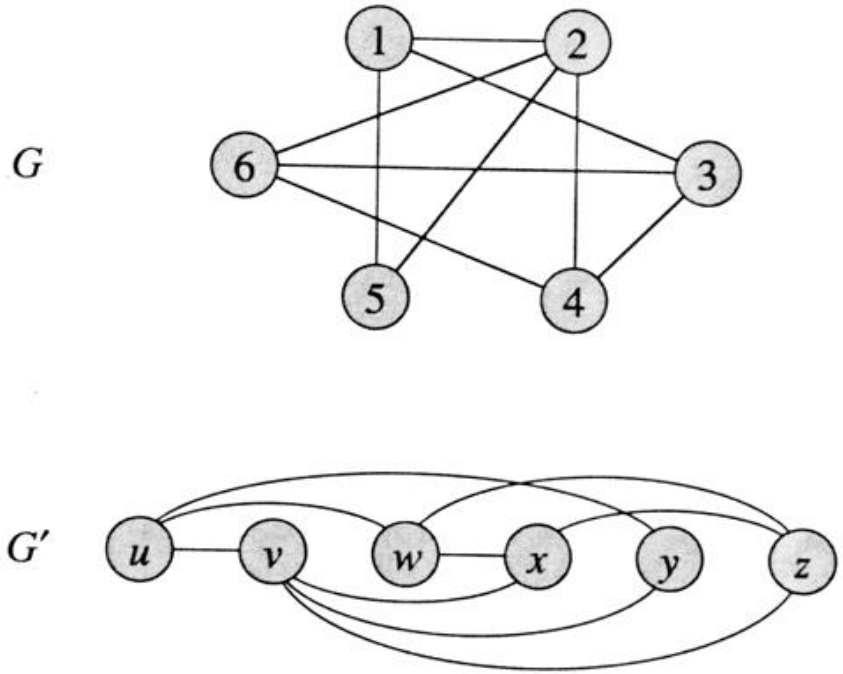
Graph G	Graph H	An isomorphism between G and H
		$f(a) = 1$ $f(b) = 6$ $f(c) = 8$ $f(d) = 3$ $f(g) = 5$ $f(h) = 2$ $f(i) = 4$ $f(j) = 7$

wikipedia



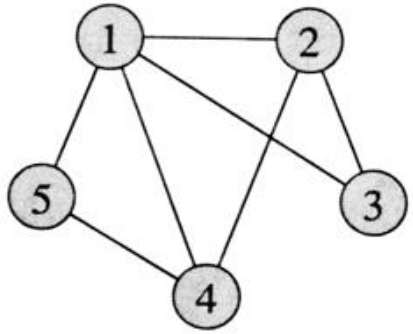
javapoint.com

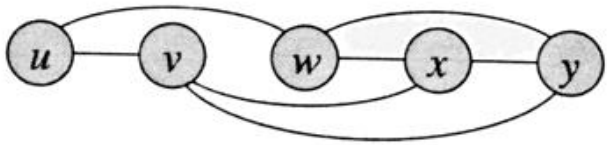
Isomorphic(?)

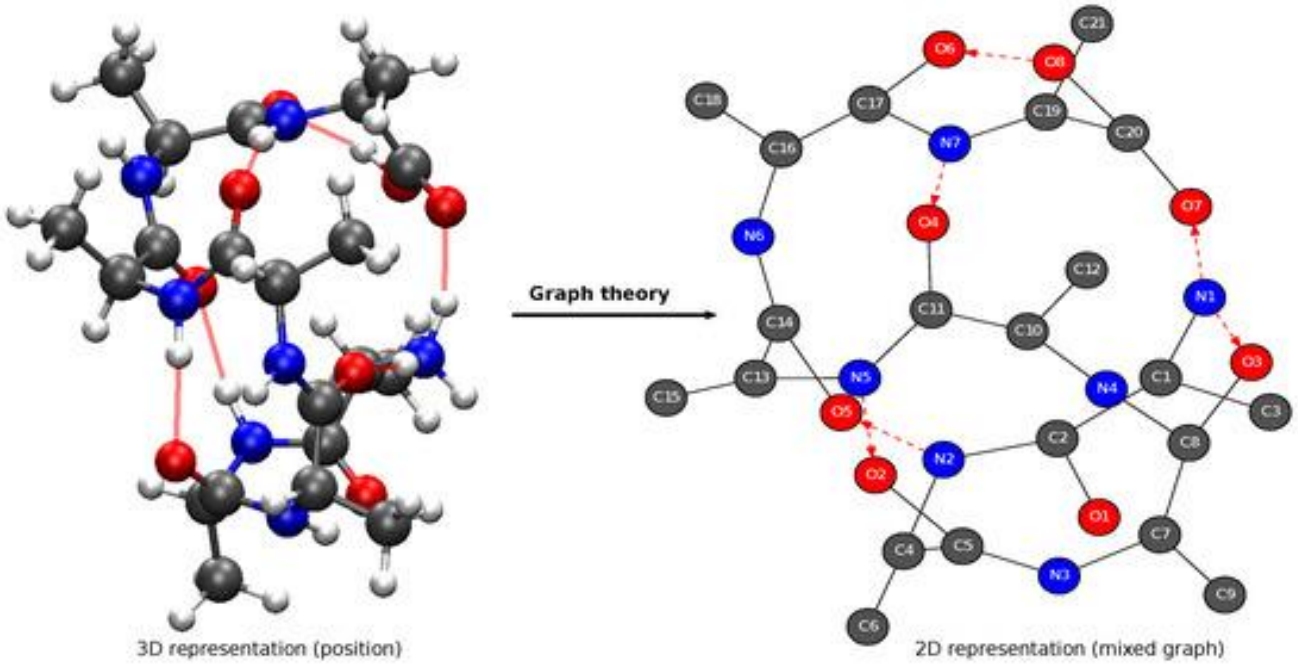


f(1)=u
f(2)=v
f(3)=w
f(4)=x
f(5)=y
f(6)=z

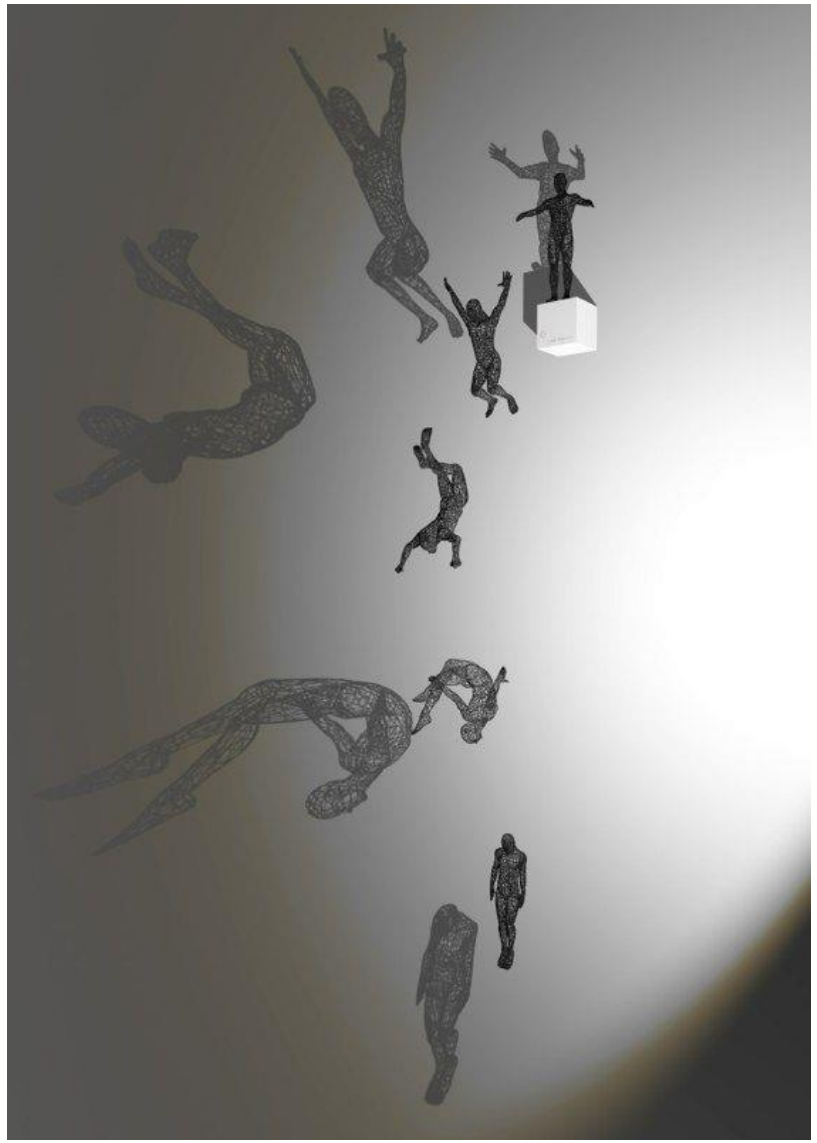
Isomorphic(?)







source

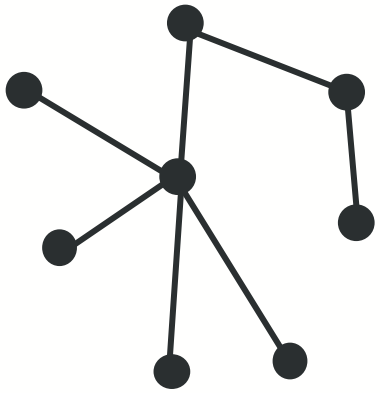


source

Real-world Applications of Graph Isomorphism

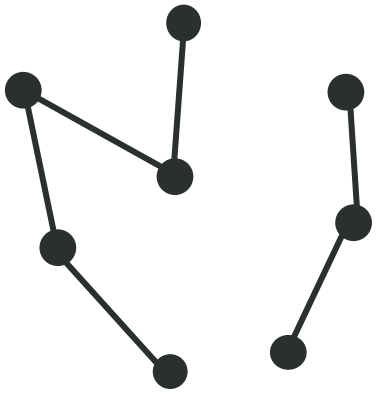
- **Shape and object matching**: In image processing, regions of an image can be modeled as graphs. Isomorphism helps match patterns, recognize objects, or detect anomalies by identifying structural similarities.
- **Code optimization**: Compilers use graph isomorphism to detect redundant code structures and optimize control flow graphs, improving program efficiency.
- **Electronic circuit verification**: Circuit designs can be represented as graphs. Graph isomorphism ensures two circuits are functionally equivalent by comparing their structural layout.
- **Community detection in Social Networks**: By modeling social structures as graphs, graph isomorphism helps identify similar communities or user behavior patterns across different networks or platforms.

Free Trees, Forests, and DAG's



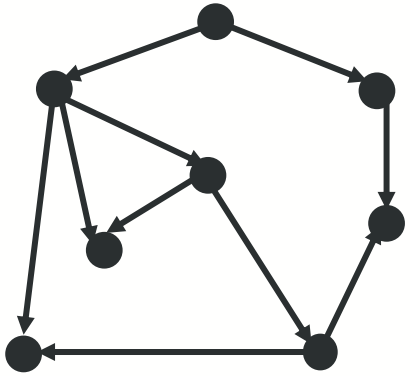
Free Tree

A free tree is an undirected graph that is connected and acyclic.



Forest

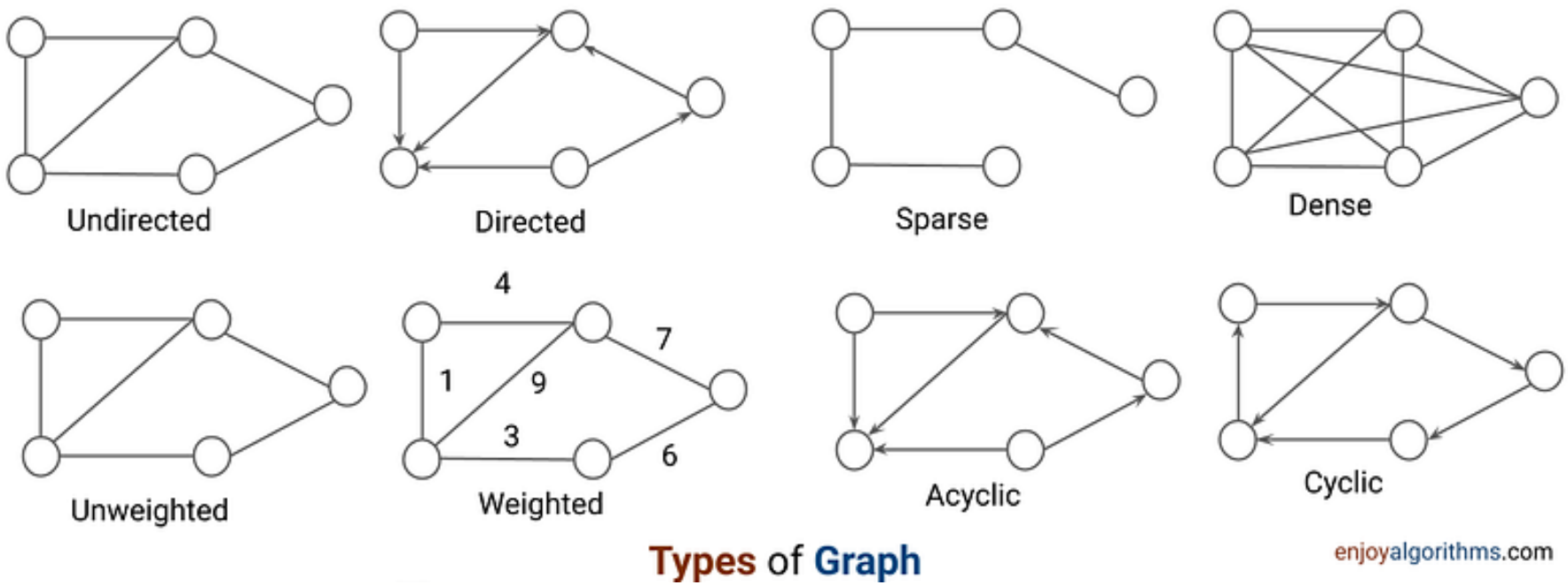
A forest is a disjoint set of trees. In other words, it's a collection of acyclic, connected components.



Directed Acyclic Graph (DAG)

A directed acyclic graph (DAG) is a directed graph with no directed cycles.

Types of Graph



Graph Representations – Adjacency Matrix

Let $G = (V, E)$ be a digraph with $n=|V|$ & $e=|E|$

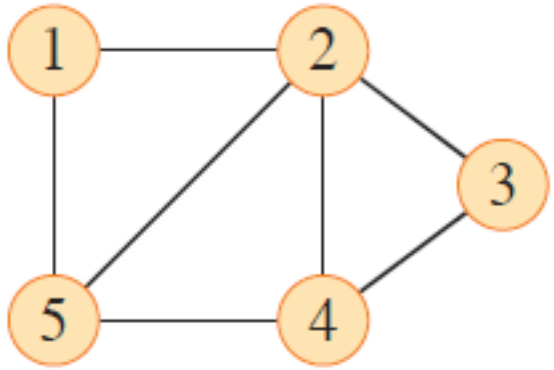
- *Adjacency Matrix*
  - Structure: An n x n matrix
  - Matrix entries: for $1 \leq v, w \leq n$

$$A[v,w] = 1 \text{ if } (v,w) \in E \text{ and } 0 \text{ otherwise}$$

  - If digraph has weights, store them in matrix instead of 1s.

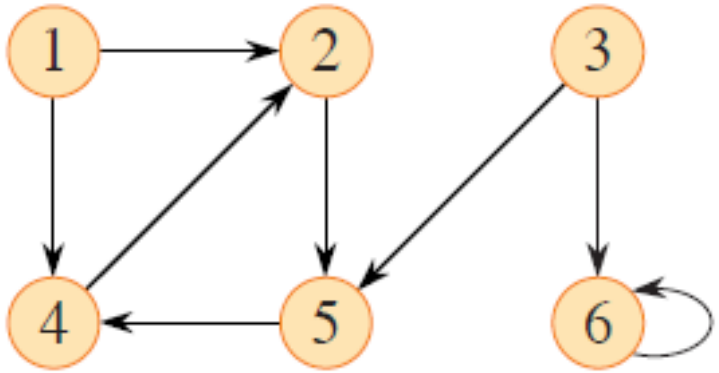
Example – Adjacency Matrix

(a)



	1	2	3	4	5
1					
2					
3			?		
4					
5					

(b)



	1	2	3	4	5	6
1						
2						
3				?		
4						
5						
6						

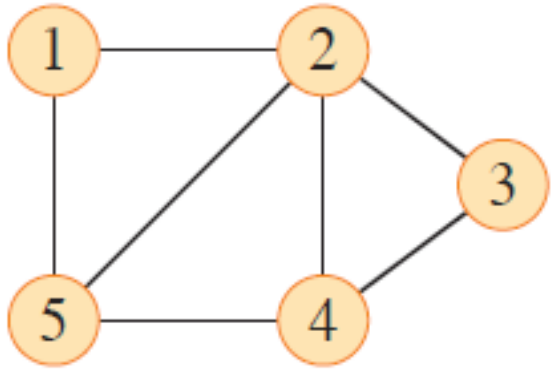
Graph Representations – Adjacency List

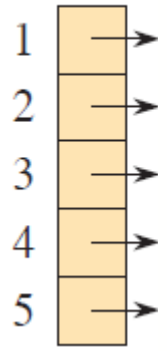
Let $G = (V, E)$ be a digraph with $n = |V|$ & $e = |E|$

- *Adjacency List*: an array $Adj[1 \ldots n]$ of pointers where for $1 \leq v \leq n$, $Adj[v]$ points to a linked list containing the vertices which are adjacent to v. If the edges have weights then they may also be stored in the linked list elements.

Example – Adjacency List

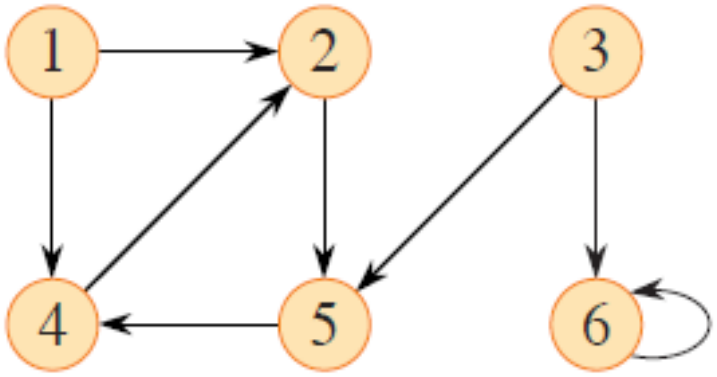
(a)

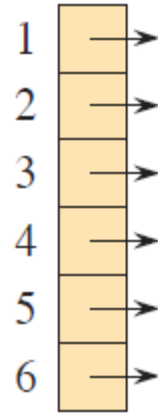




?

(b)





?

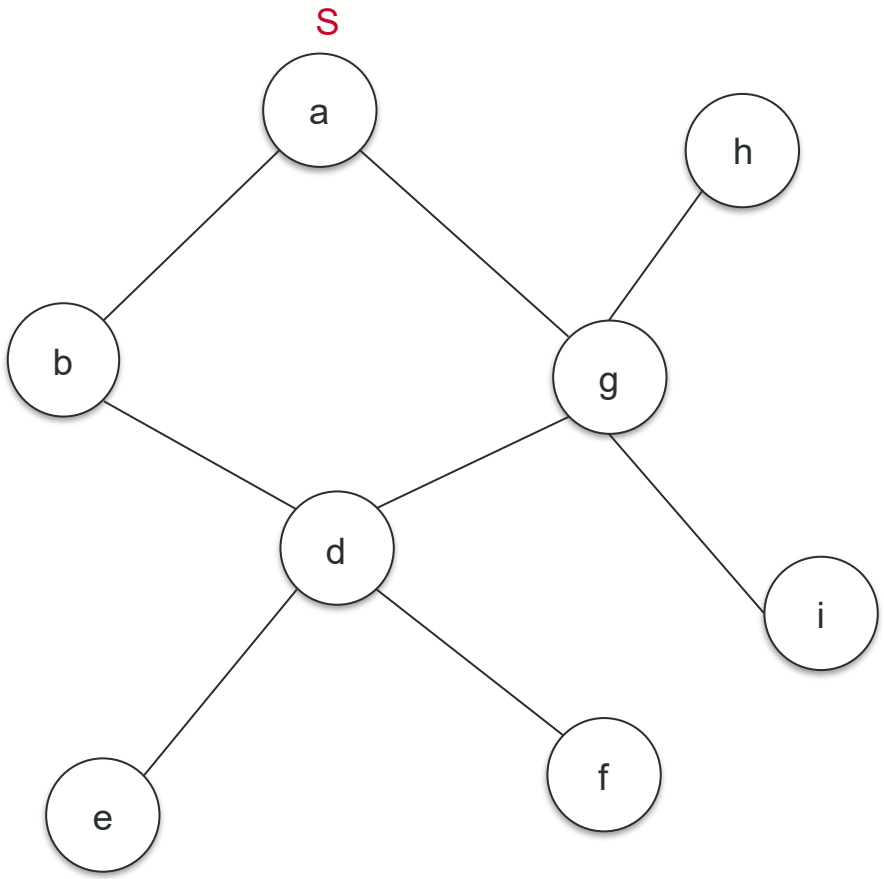
Graph as an ADT

- A graph consists of nodes (vertices) and edges and supports various operations.
- Basic Operations:
  - **adjacent(G, x, y)**: Check if there's an edge from vertex x to vertex y.
  - **neighbors(G, x)**: List all vertices y connected to vertex x.
  - **add_vertex(G, x)**: Add vertex x if it doesn't exist.
  - **remove_vertex(G, x)**: Remove vertex x if it exists.
  - **add_edge(G, x, y, z)**: Add edge z from vertex x to vertex y if it doesn't exist.
  - **remove_edge(G, x, y)**: Remove the edge from vertex x to vertex y if it exists.
  - **get_vertex_value(G, x)**: Retrieve the value associated with vertex x.
  - **set_vertex_value(G, x, v)**: Set the value of vertex x to v.
- Edge Value Operations:
  - **get_edge_value(G, x, y)**: Retrieve the value associated with edge (x,y)(x,y).
  - **set_edge_value(G, x, y, v)**: Set the value of edge (x,y)(x,y) to vv.

BFS

Finding Shortest Paths

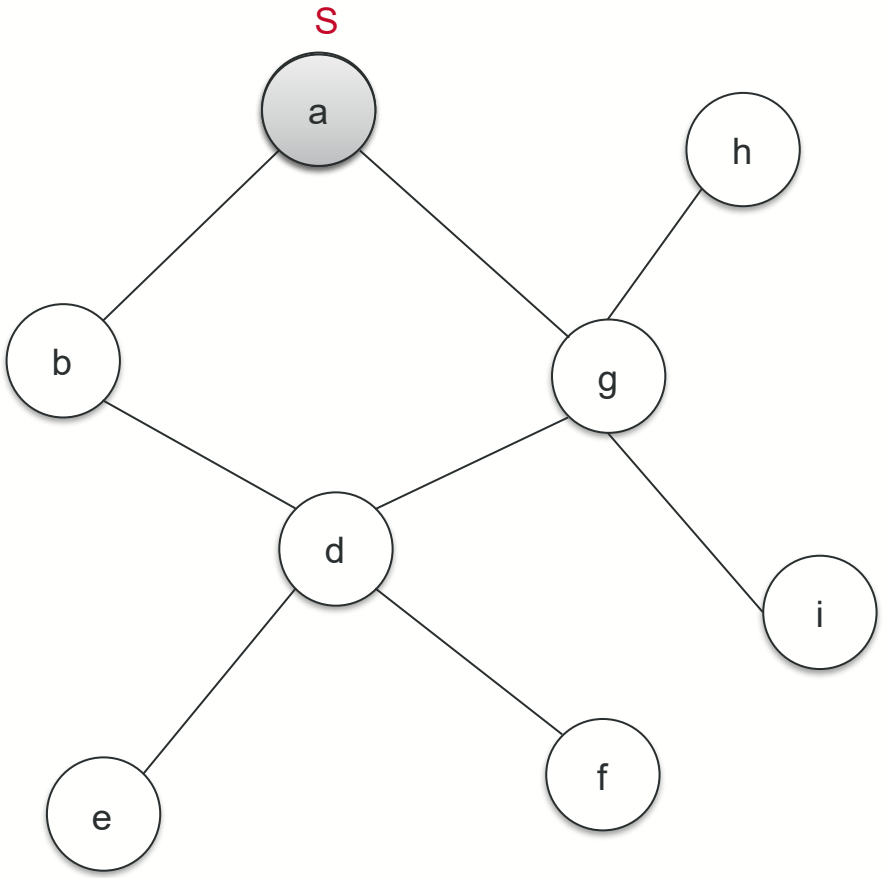
- Given an undirected graph and source vertex s, the length of a path in a graph (without edge weights) is the number of edges on the path. Find the shortest path from s to each other vertex in the graph.



Breadth-First-Search (BFS)

- Given:
  - $G = (V, E)$
  - A distinguished *source* vertex
- Systematically explores the edges of G to discover every vertex that is reachable from s
  - Computes (shortest) distance from s to all reachable vertices
  - Produces a *breadth-first-tree* with root s that contains all reachable vertices

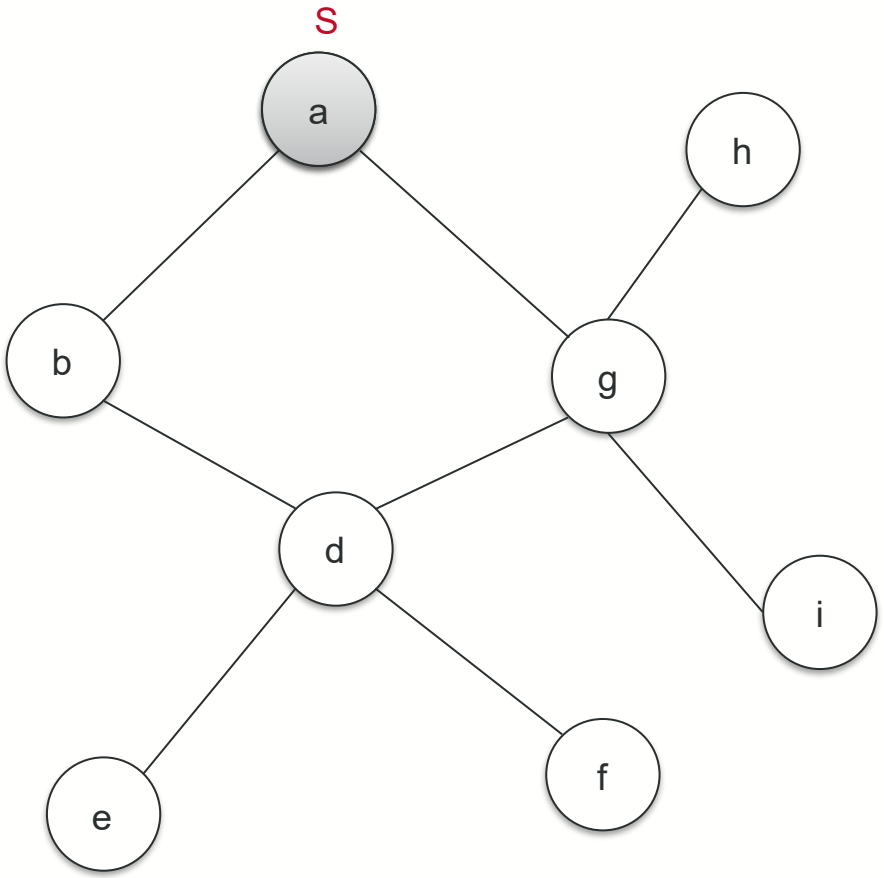
BFS example



Discovered but not yet explored

Explored (adjacent nodes are expanded):

BFS example

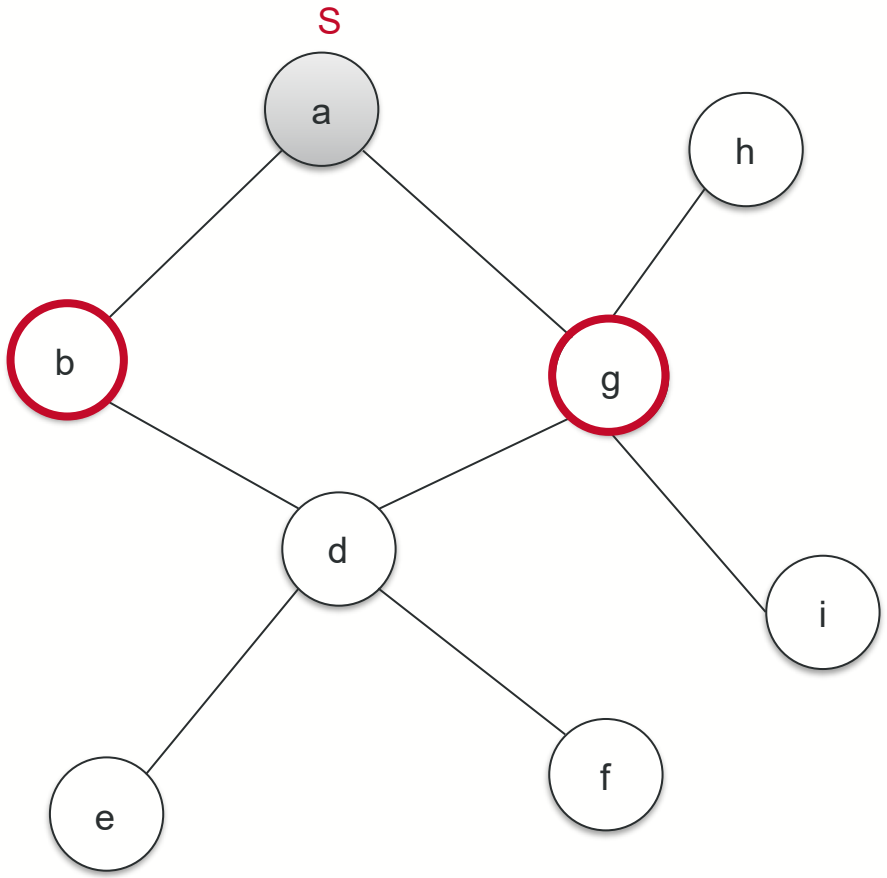


Discovered but not yet explored

a									

Explored (adjacent nodes are expanded):

BFS example

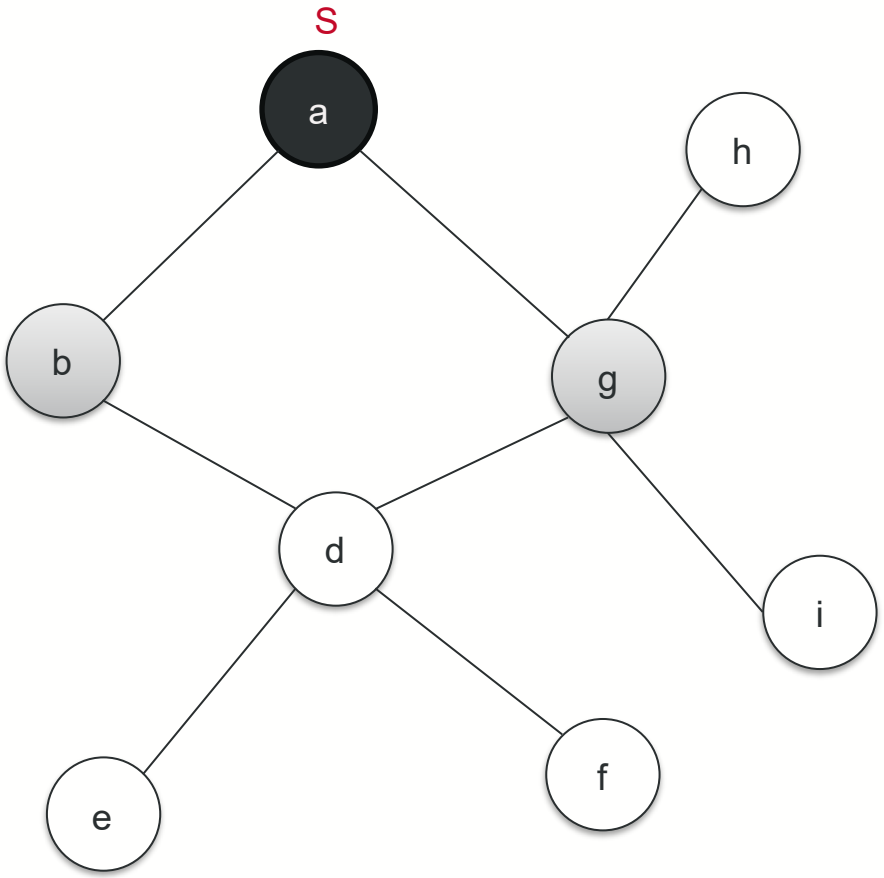


Discovered but not yet explored

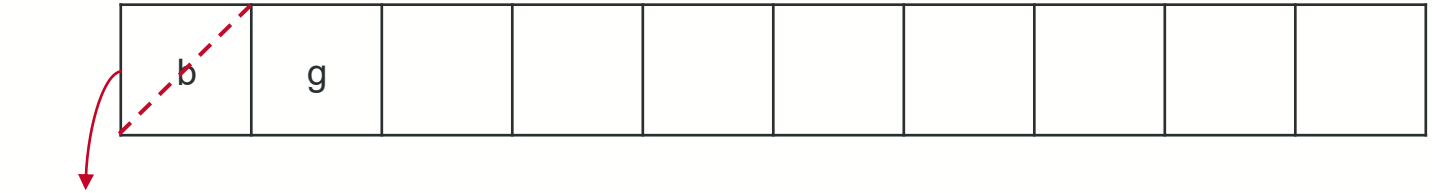
Explore a → b and g found.

Explored (adjacent nodes are expanded):

BFS example



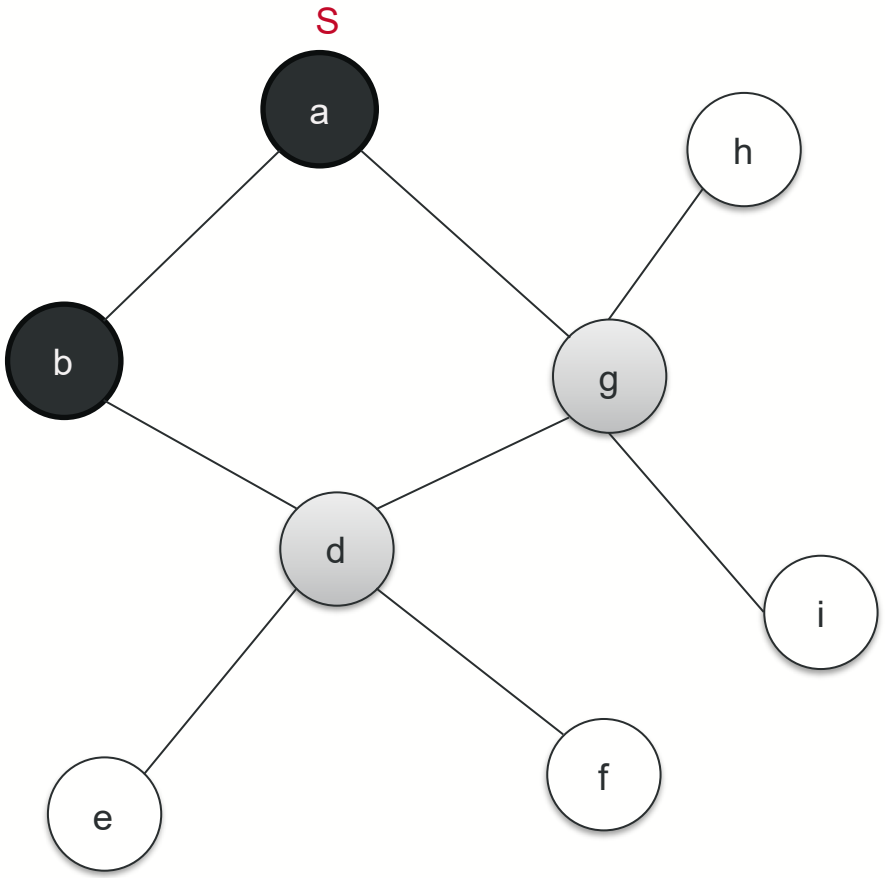
Discovered but not yet explored



Explore b → d found

Explored (adjacent nodes are expanded):

BFS example



Discovered but not yet explored

g	d								

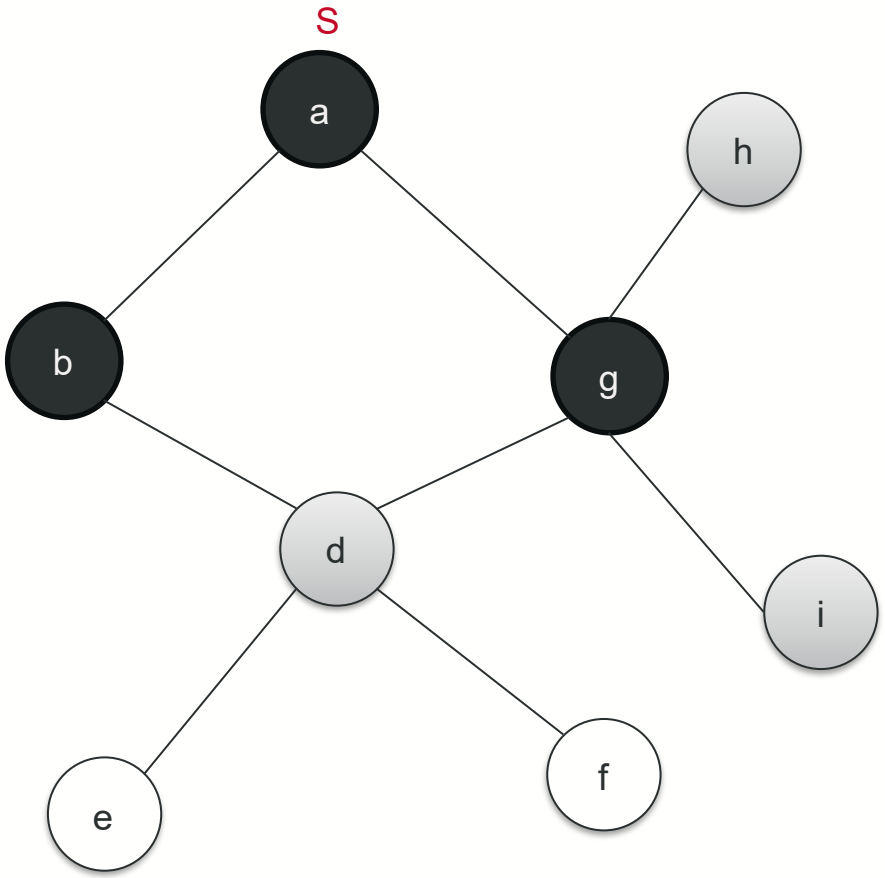
Explore g → found a, d, h, i.

a is already explored. d is already discovered. Add h and i to the list.

Explored (adjacent nodes are expanded):

a	b								

BFS example



Discovered but not yet explored

d	h	i							

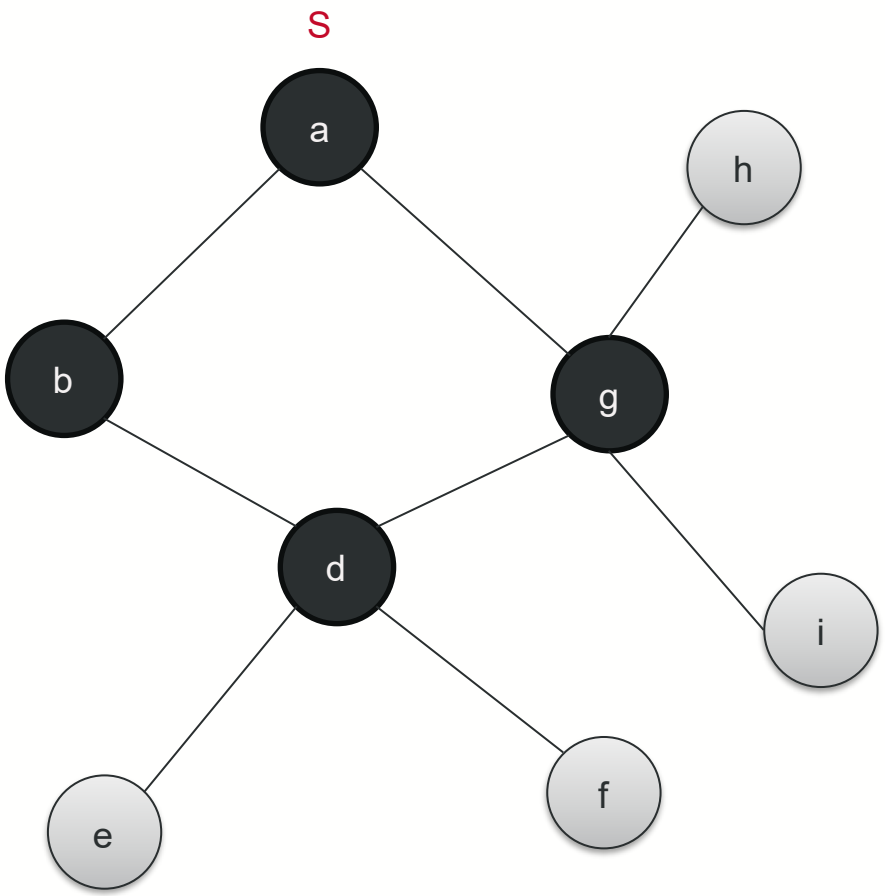
Explore d → found b, g, e, f. b is already explored. g is already discovered.

Add e and f to the list

Explored (adjacent nodes are expanded):

a	b	g							

BFS example



Discovered but not yet explored

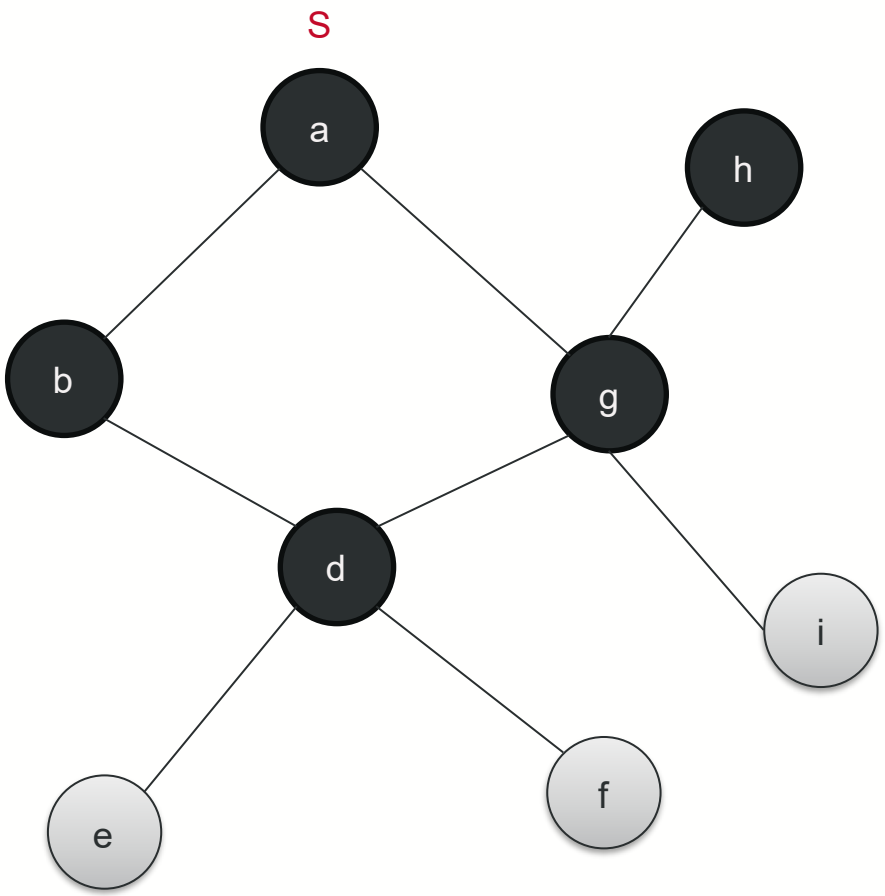
h	i	e	f						

Explore h → g is already explored

Explored (adjacent nodes are expanded):

a	b	g	d						

BFS example



Discovered but not yet explored

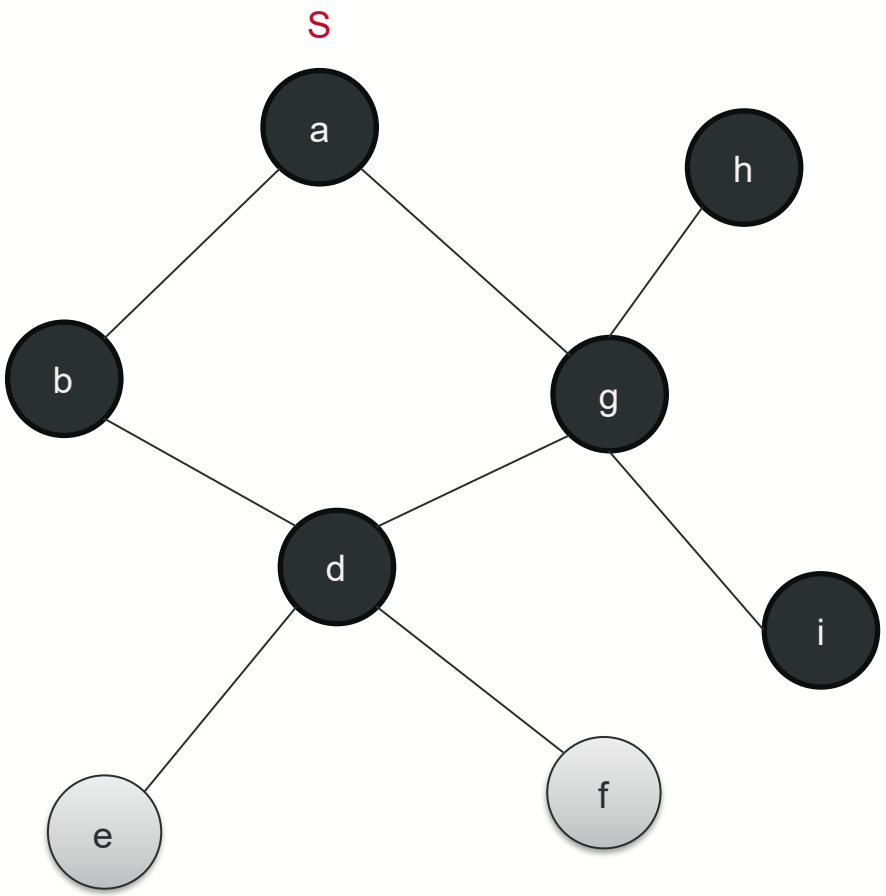
i	e	f							

Explore i → g is already explored.

Explored (adjacent nodes are expanded):

a	b	g	d	h					

BFS example



Discovered but not yet explored

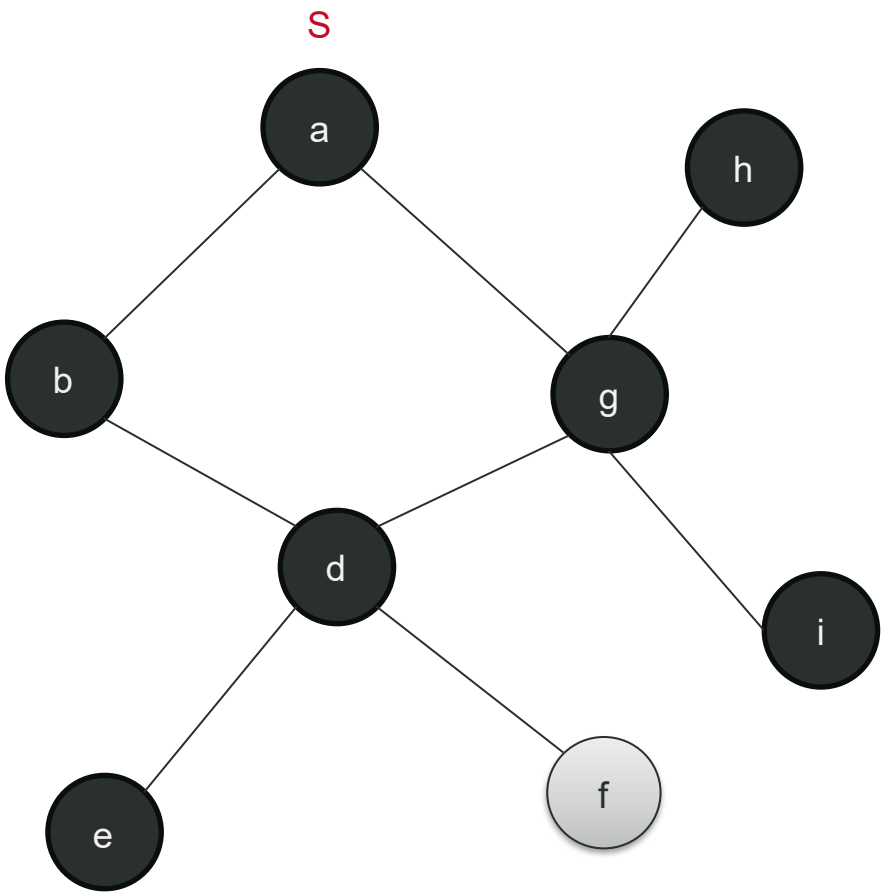
e	f								

Explore e → d is already explored.

Explored (adjacent nodes are expanded):

a	b	g	d	h	i				

BFS example



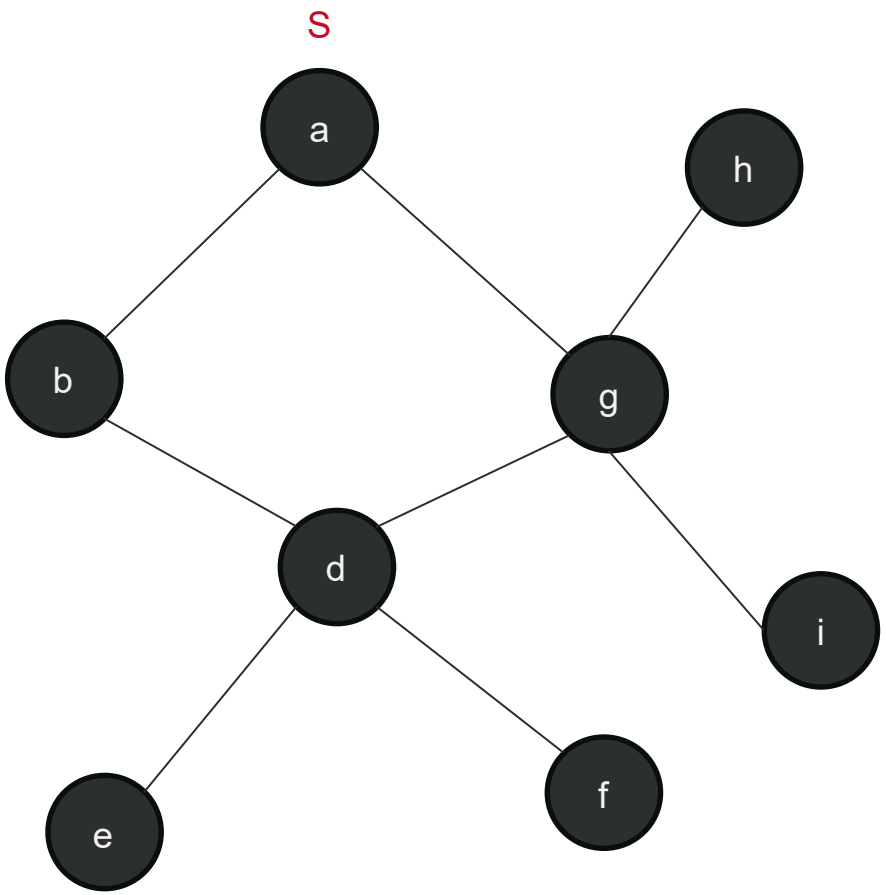
Discovered but not yet explored

Explore f → d is already explored.

Explored (adjacent nodes are expanded):

a	b	g	d	h	i	e			

BFS example



What data structure is this?

- FIFO (First in, first out)
- Queue

Discovered but not yet explored

No more nodes to explore. End the traversal.

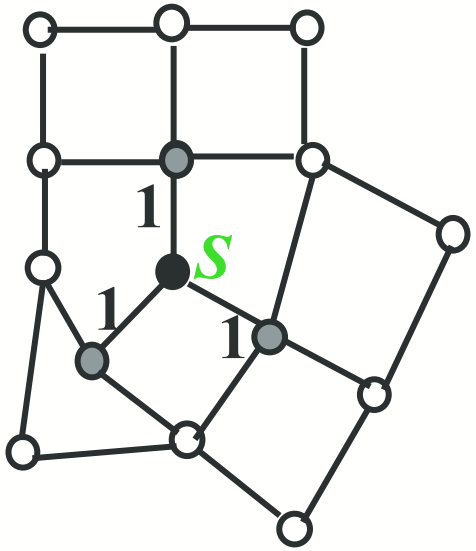
Explored (adjacent nodes are expanded):

a	b	g	d	h	i	e	f		

Breadth-First-Search (BFS)

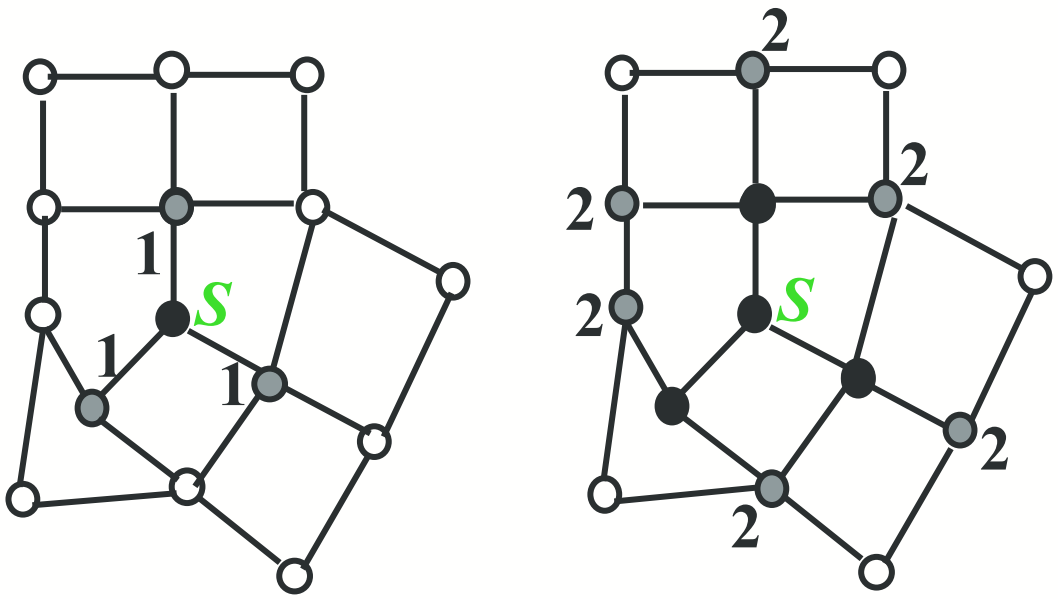
- BFS colors each vertex:
  - White: undiscovered
  - Gray: discovered but “not done yet”
  - Black: all adjacent vertices have been discovered

BFS for Shortest Paths



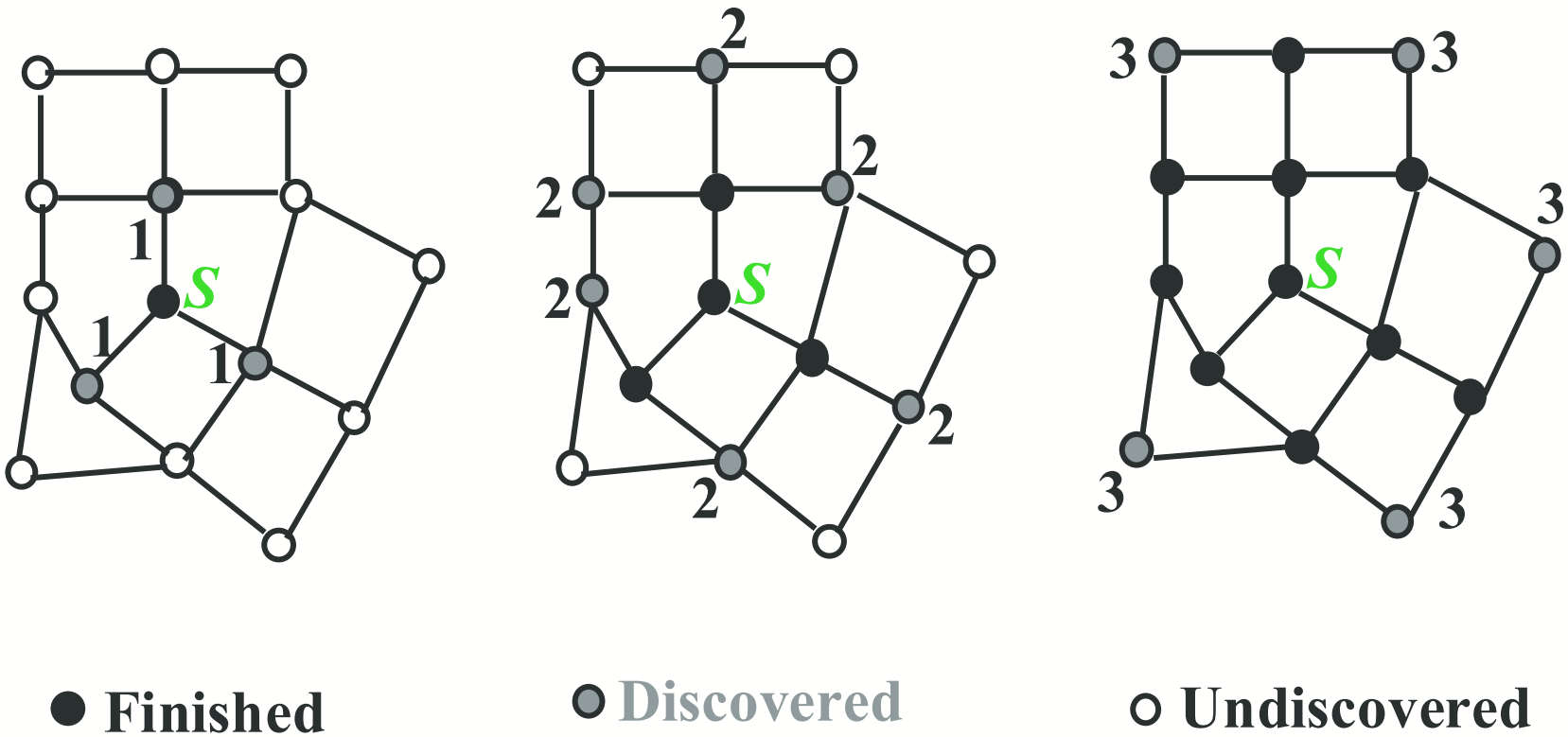
● Finished ○ Discovered ○ Undiscovered

BFS for Shortest Paths



● Finished ○ Discovered ○ Undiscovered

BFS for Shortest Paths

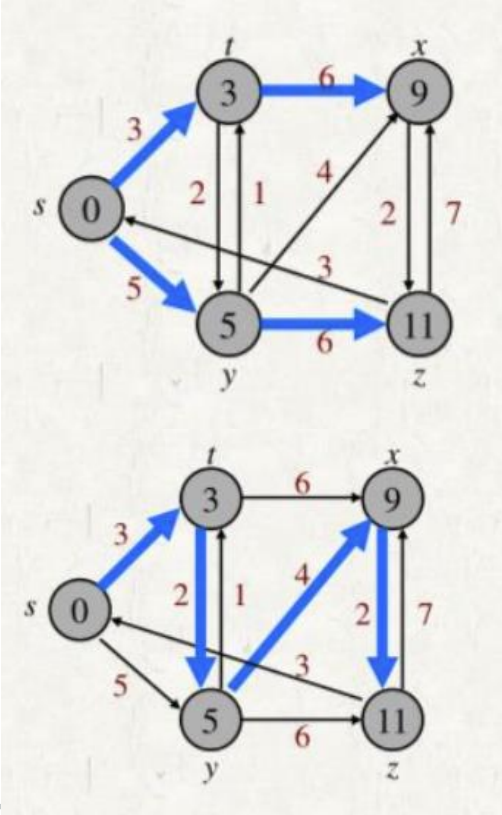


Breadth-First Tree (1)

- A vertex u is called the predecessor of vertex v, denoted as $\pi[v]$, if there is a directed edge from u to v in the path
- For a graph $G = (V, E)$ with source s, the predecessor subgraph of G is $G_\pi = (V_\pi, E_\pi)$ where
  - $V_\pi = \{v \in V : \pi[v] \neq NIL\} \cup \{s\}$
  - $E_\pi = \{(\pi[v], v) \in E : v \in V_\pi - \{s\}\}$
  - i.e., a subgraph formed by the predecessors of vertices reachable from the source s.
- Each vertex v (except s) has exactly one predecessor ($\pi[v]$)
- if a vertex v was never discovered during traversal (e.g., disconnected from s), then $\pi[v] = NIL$, and that vertex is excluded.

Image:hanyang university

- $V_\pi = \{v \in V : \pi[v] \neq NIL\} \cup \{s\}$
- $E_\pi = \{(\pi[v], v) \in E : v \in V_\pi - \{s\}\}$



$V_\pi = \{s, t, x, y, z\}$

$E_\pi = \{(s, t), (s, y), (t, x), (y, z)\}$

$V_\pi = \{s, t, x, y, z\}$

$E_\pi = \{(s, t), (t, y), (y, x), (x, z)\}$

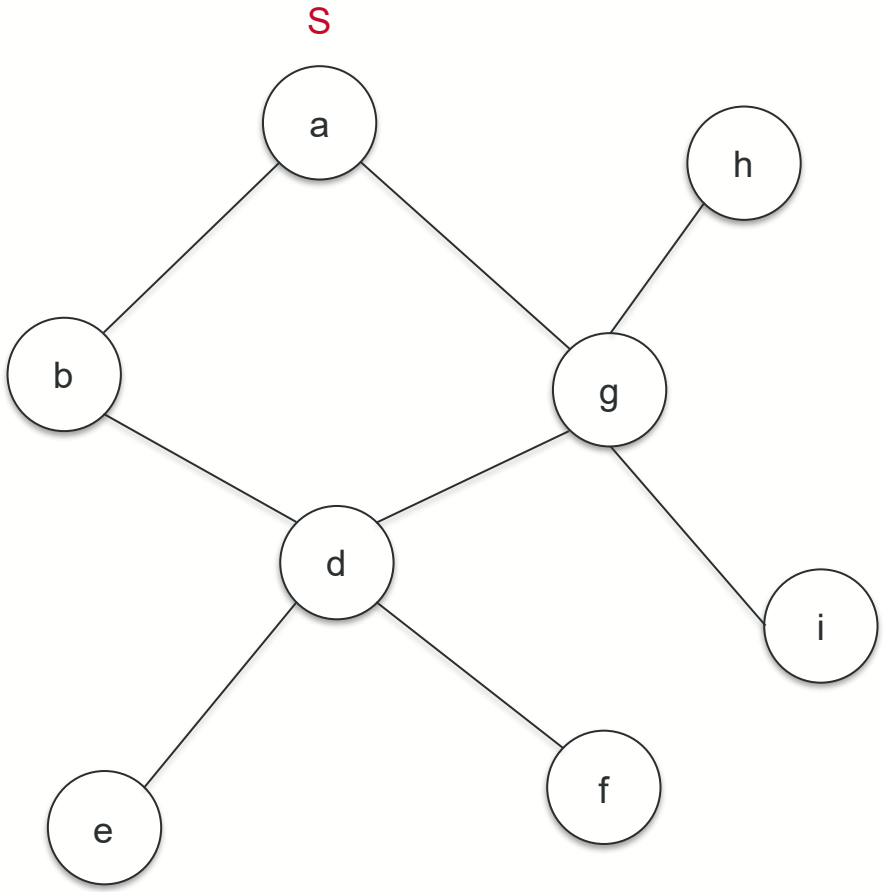
Breadth-First Tree (2)

- The predecessor subgraph G_π is a breadth-first tree if:
  - V_π consists of the vertices reachable from s and
  - for all $v \in V_\pi$, there is a **unique simple path** from s to v in G_π that is also a **shortest path** from s to v in G.
- The edges in E_π are called *tree edges of the form* $(\pi[v], v)$, forming a parent-child hierarchy
- For n vertices, a tree has n-1 edges. Thus $|E_\pi| = |V_\pi| - 1$

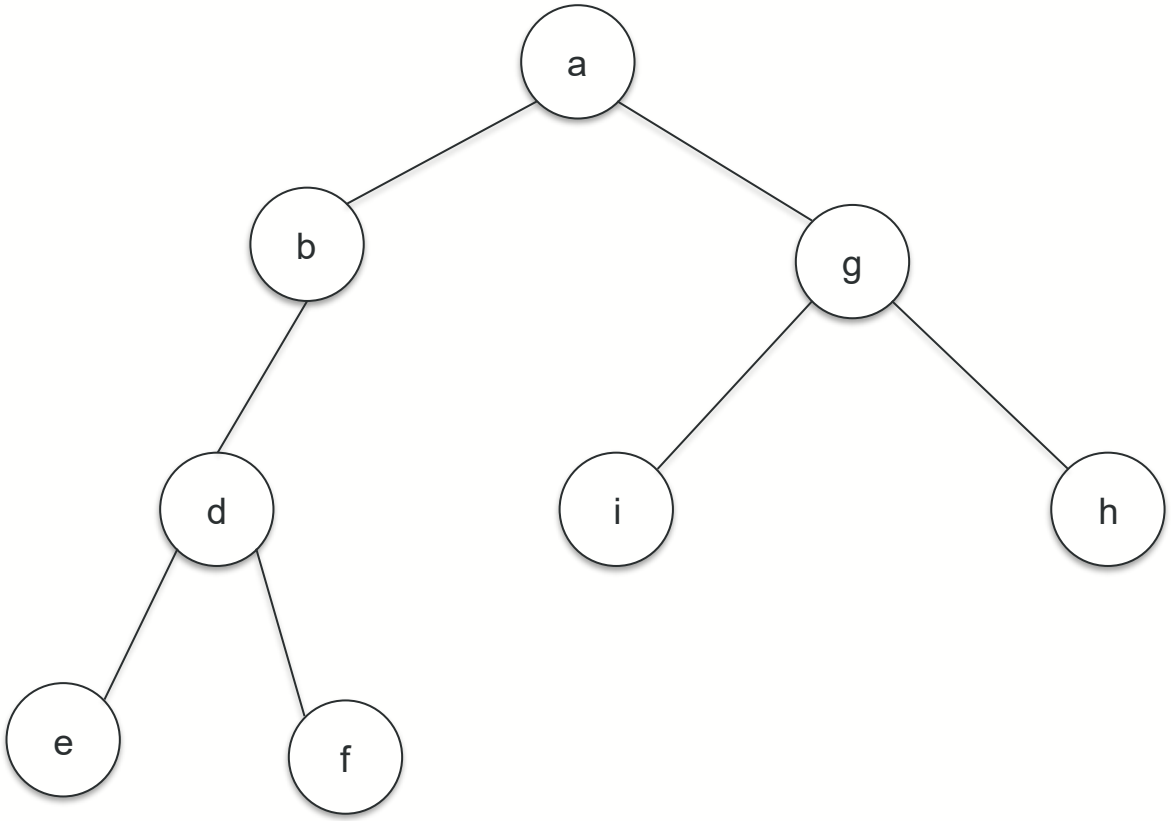
Intuition: Breadth-First Tree

- Predecessor Pointers record which vertex discovered which during BFS
- These pointers form an inverted tree (edges point from child→parent)
  - This structure is an acyclic directed graph with the source vertex as the root
  - Every other vertex has a unique path to the root
- If you ignore direction (make edges undirected), the structure becomes a BFS tree
  - The BFS tree organizes vertices into levels based on their distance from the source
- Different BFS trees can arise depending on:
  - the chosen source
  - the order in which neighbors are enqueued
- Tree edges are the edges of the original graph that connect each vertex to its predecessor in the BFS tree

BFS example – Breadth First Tree



BFS tree is built dynamically during the traversal

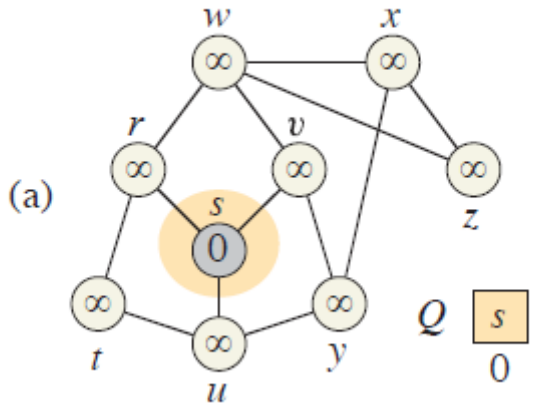


```
BFS(G,s)
1.  for each vertex u in (V[G] - {s})
2               do color[u] ← white
3               d[u] ← ∝
4               π[u] ← nil
5   color[s] ← gray
6   d[s] ← 0
7   π[s] ← nil
8   Q ← ∅
9   enqueue(Q,s)
10  while Q ≠ ∅
11              do u ← dequeue(Q)
12                      for each v in Adj[u]
13                              do if color[v] = white
14                                      then color[v] ← gray
15                                      d[v] ← d[u] + 1
16                                      π[v] ← u
17                                      enqueue(Q,v)
18                      color[u] ← black
```

white: undiscovered
gray: discovered
black: finished

Q: a queue of discovered vertices
color[v]: color of v
d[v]: distance from s to v
π[u]: predecessor of u

Operations of BFS on a Graph



initialize

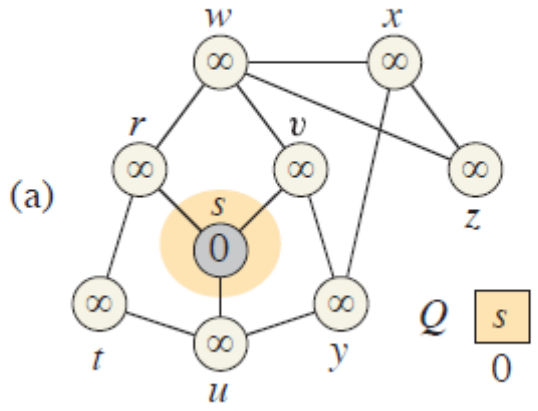
```
1.      for each vertex u in (V[G] - {s})
2           do color[u] ← white
3           d[u] ← ∝
4           π[u] ← nil
```

	r	s	t	u	v	w	x	y	z
d	∝	∝	∝	∝	∝	∝	∝	∝	∝
π	nil	nil	nil	nil	nil	nil	nil	nil	nil
c	w	w	w	w	w	w	w	w	w

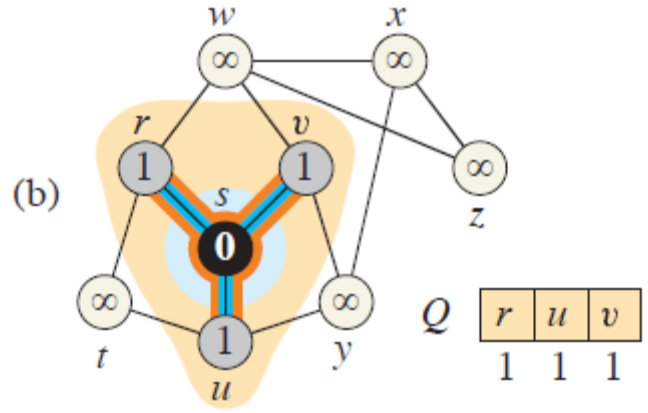
```
5       color[s] ← gray
6       d[s] ← 0
7       π[s] ← nil
8       Q ← ∅
9       enqueue(Q,s)
```

	r	s	t	u	v	w	x	y	z
d	∝	0	∝	∝	∝	∝	∝	∝	∝
π	nil	nil	nil	nil	nil	nil	nil	nil	nil
c	w	g	w	w	w	w	w	w	w

Operations of BFS on a Graph

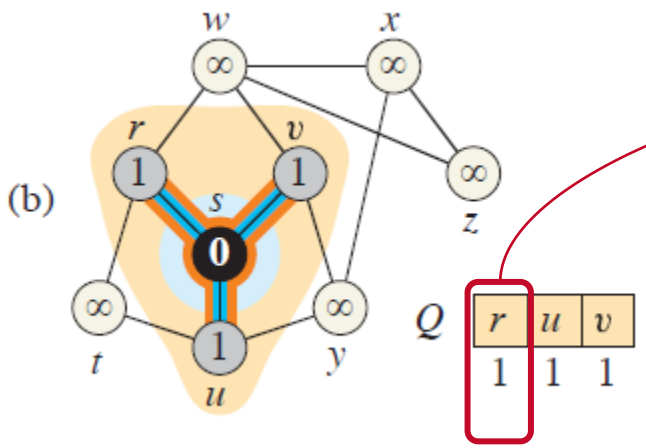



```
10  while Q ≠ ∅
11     do u ← dequeue(Q)
12           for each v in Adj[u]
13              do if color[v] = white
14              then color[v] ← gray
15                   d[v] ← d[u] + 1
16                   π[v] ← u
17                   enqueue(Q,v)
18           color[u] ← black
```

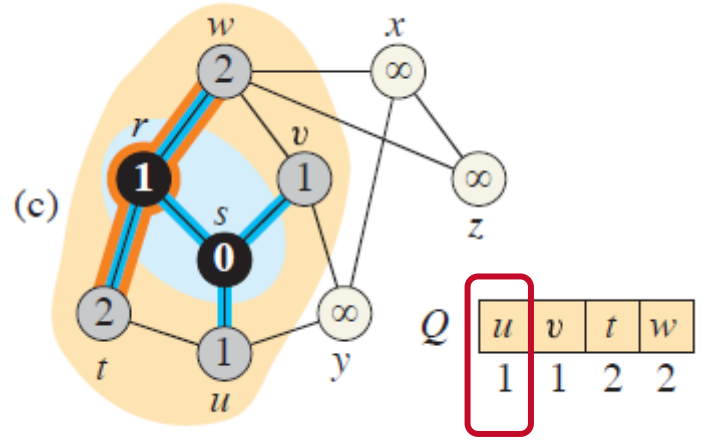


	r	s	t	u	v	w	x	y	z
d	1	0	∝	1	1	∝	∝	∝	∝
π	s	nil	nil	s	s	nil	nil	nil	nil
c	g	b	w	g	g	w	w	w	w

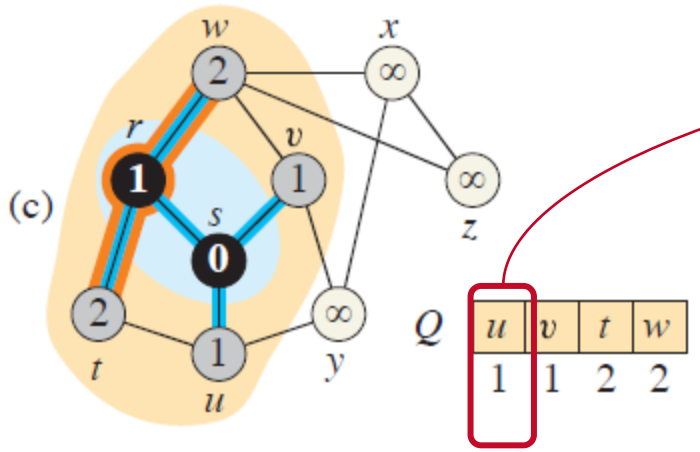
Operations of BFS on a Graph



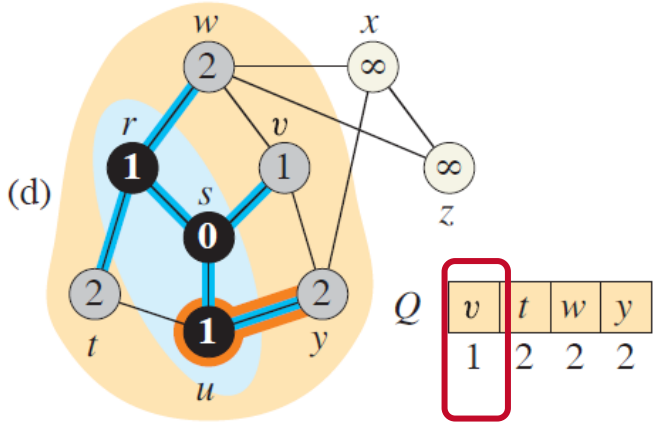

```
10 while Q ≠ ∅
11       do u ← dequeue(Q)
12          for each v in Adj[u]
13             do if color[v] = white
14                   then color[v] ← gray
15                   d[v] ← d[u] + 1
16                    π[v] ← u
17                   enqueue(Q,v)
18          color[u] ← black
```



	r	s	t	u	v	w	x	y	z
d	1	0	2	1	1	2	∝	∝	∝
π	s	nil	r	s	s	r	nil	nil	nil

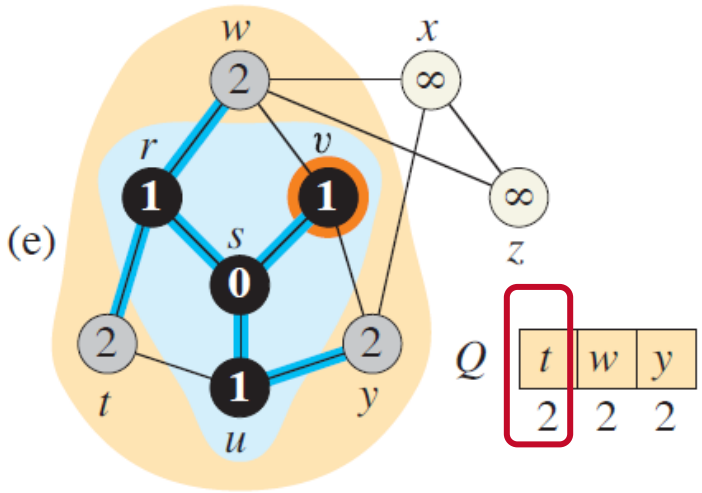


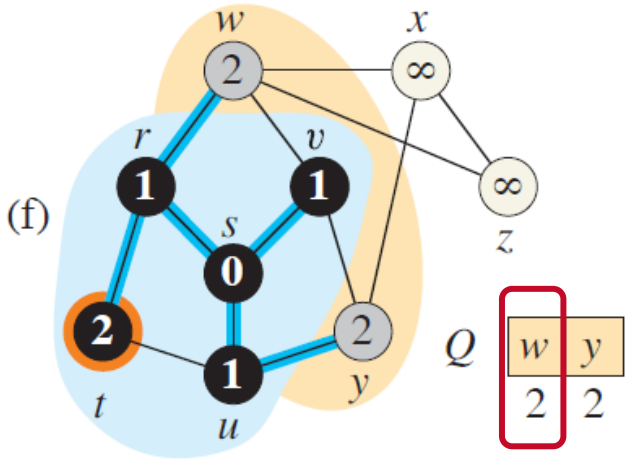

```
10 while Q ≠ ∅
11       do u ← dequeue(Q)
12                for each v in Adj[u]
13                      do if color[v] = white
14                              then color[v] ← gray
15                              d[v] ← d[u] + 1
16                               π[v] ← u
17                              enqueue(Q,v)
18                color[u] ← black
```



	r	s	t	u	v	w	x	y	z
d	1	0	2	1	1	2	∞	2	∞
π	s	nil	r	s	s	r	nil	u	nil

Operations of BFS on a Graph

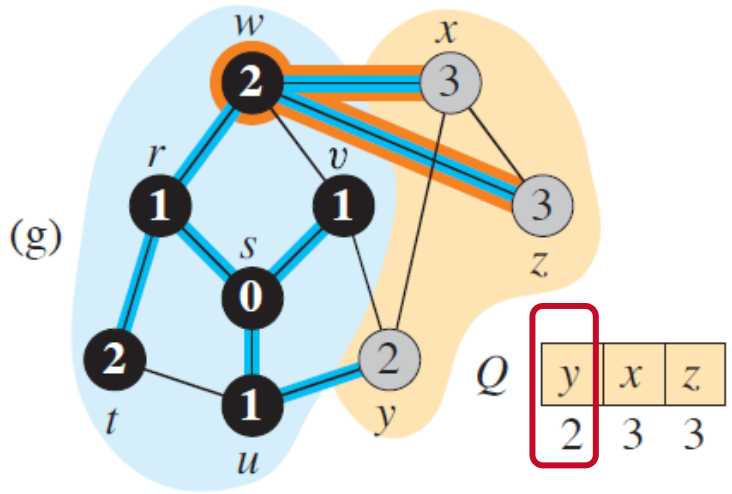


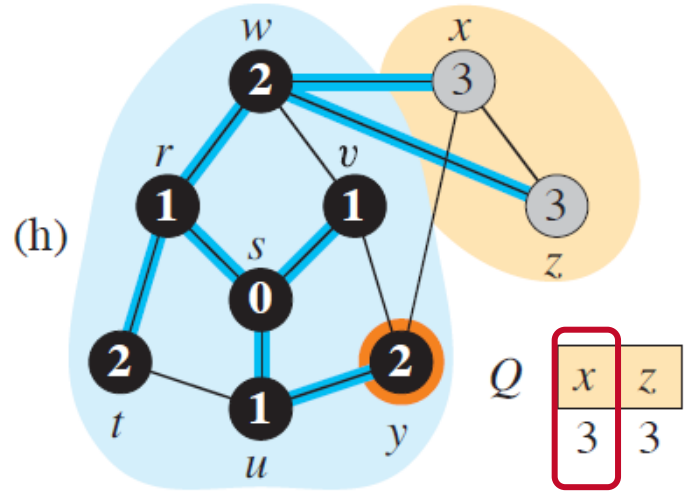


	r	s	t	u	v	w	x	y	z
d	1	0	2	1	1	2	∝	2	∝
π	s	nil	r	s	s	r	nil	u	nil

	r	s	t	u	v	w	x	y	z
d	1	0	2	1	1	2	∝	2	∝
π	s	nil	r	s	s	r	nil	u	nil

Operations of BFS on a Graph

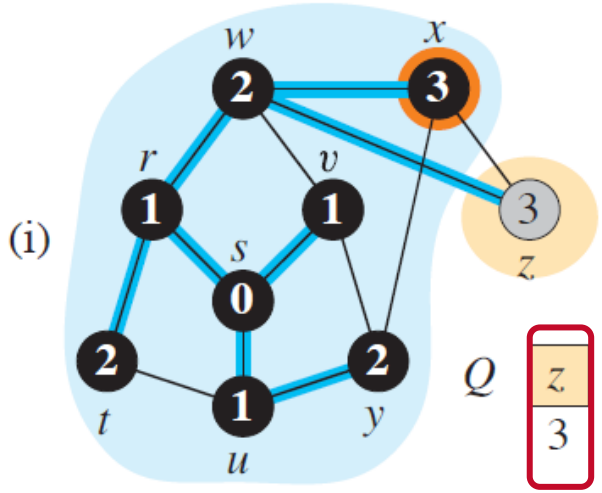


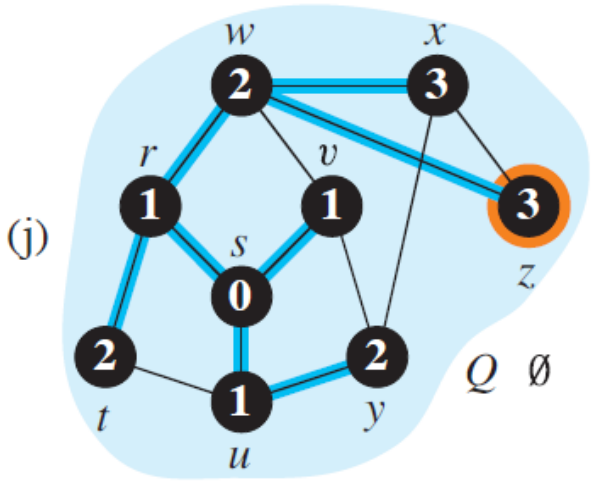


	r	s	t	u	v	w	x	y	z
d	1	0	2	1	1	2	3	2	3
π	s	nil	r	s	s	r	w	u	w

	r	s	t	u	v	w	x	y	z
d	1	0	2	1	1	2	3	2	3
π	s	nil	r	s	s	r	w	u	w

Operations of BFS on a Graph





	r	s	t	u	v	w	x	y	z
d	1	0	2	1	1	2	3	2	3
π	s	nil	r	s	s	r	w	u	w

	r	s	t	u	v	w	x	y	z
d	1	0	2	1	1	2	3	2	3
π	s	nil	r	s	s	r	w	u	w

Analysis of BFS

- Initialization takes O(V).
- Traversal Loop
  - After initialization, each vertex is enqueued and dequeued at most once, and each operation takes O(1). So, total time for queuing is O(V).
  - The adjacency list of each vertex is scanned at most once. The sum of lengths of all adjacency lists is Θ(E).
    - Directed graph: each edge is counted once because it has a specific direction
    - Undirected graph: each edge is counted twice (once for each vertex it connects)
- Summing up over all vertices => total running time of BFS is O(V+E), linear in the size of the adjacency list representation of graph.

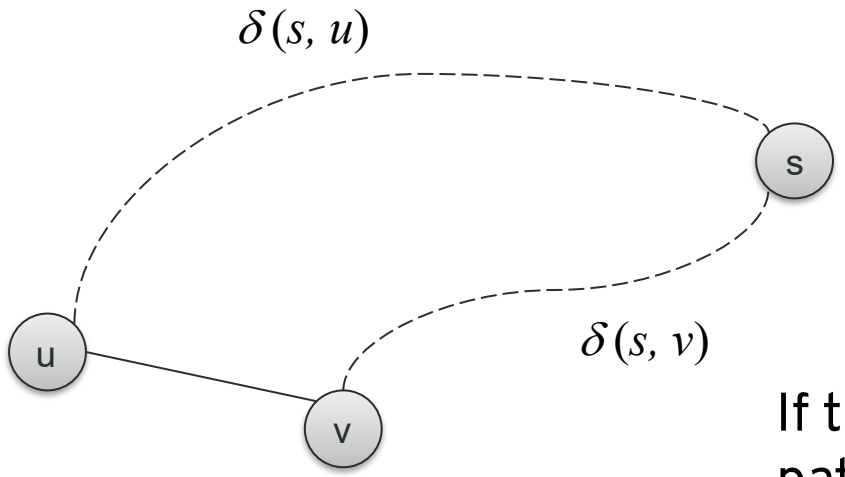
Shortest Paths

- *Shortest-Path distance* $\delta(s, v)$ from vertex s to vertex v is the minimum number of edges in any path from s to v, or else ∞ if there is no path from s to v.

- A path from s to v with length equal to $\delta(s, v)$ is called a *shortest path*.

Lemmas (1)

- Let $G = (V, E)$ be a directed or undirected graph, and let $s \in V$ be an arbitrary vertex. Then, for any edge $(u,v) \in E$, $\delta(s, v) \leq \delta(s, u) + 1$.



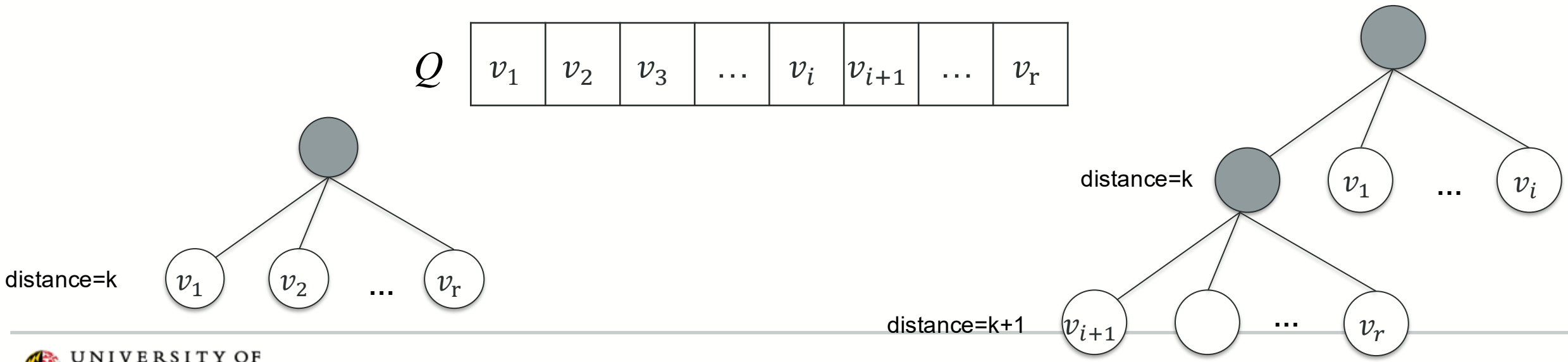
If there is an edge from u to v, then the shortest path to v is at most one more than the shortest path to u.

Lemmas (2)

- Let $G = (V, E)$ be a directed or undirected graph, and suppose that BFS is run on G from a given source vertex $s \in V$. Then upon termination, for each vertex $v \in V$, the value $d[v]$ computed by BFS satisfies $d[v] \leq \delta(s, v)$.
  - $d[v]$: distance (number of edges) from s to v computed by BFS.
  - $\delta(s, v)$: true shortest-path distance between s and v in the graph.
  - Proof using induction using Lemma 1
  - BFS's computed distance for each vertex is never greater than the true shortest-path distance

Lemmas (3)

- BFS visits vertices in increasing distance order.
- Suppose that during the execution of BFS on a graph G, the queue Q contains vertices $(v_1, \ldots, v_r)$, where v_1 is the head of Q and v_r is the tail. Then, $d[v_r] \leq d[v_1] + 1$ and $d[v_i] \leq d[v_i + 1]$ for $i = 1, 2, \ldots, r - 1$.
  - Distances in the queue never decrease. They either stay the same or increase.



Lemmas (3) contd.

- Once a vertex is dequeued, its d[v] is finalized.
- BFS never revisits a vertex once it's processed.
- The first time BFS reaches a vertex, it has found the shortest path to it.

Correctness of BFS

- Let $G = (V, E)$ be a directed or undirected graph, and suppose that BFS is run on G from a given source vertex $s \in V$. Then, during its execution, BFS discovers every vertex $v \in V$ that is reachable from the source s, and upon termination, $d[v] = \delta(s, v)$ for all $v \in V$. Moreover, for any vertex $v \neq s$ that is reachable from s, one of the shortest paths from s to v is the shortest path from s to $\pi[v]$ followed by the edge $(\pi[v], v)$.

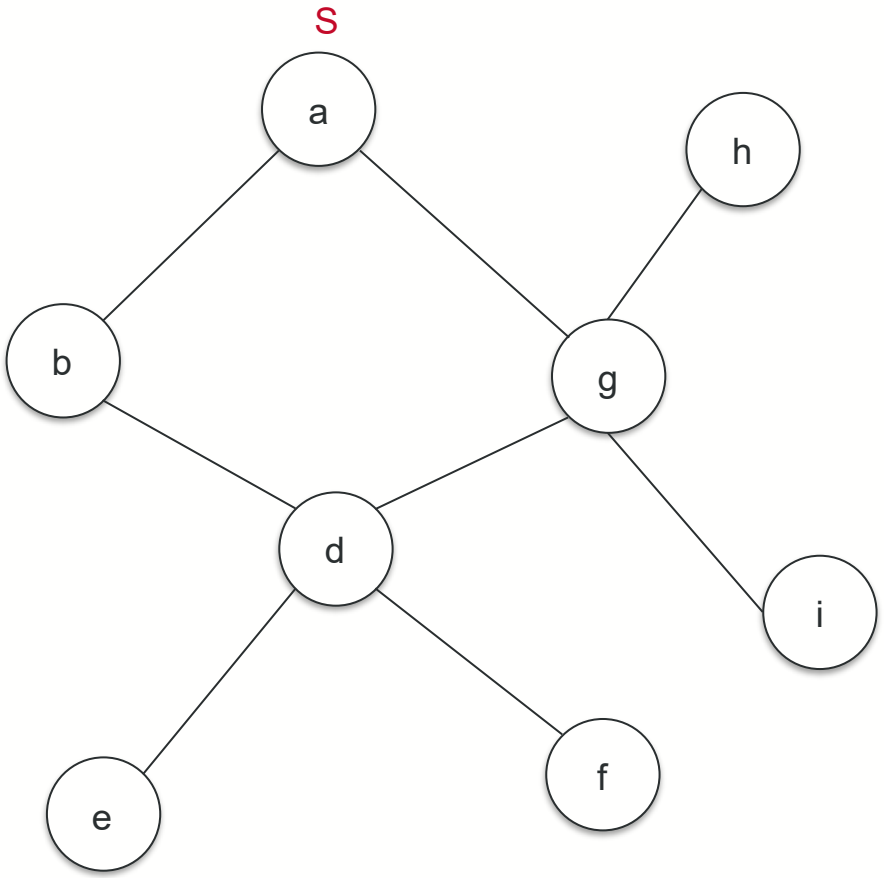
- For more details, refer to Chapter 20 of CLRS (4th ed).

DFS

Depth-First-Search (DFS)

- Explore edges out of the most recently discovered vertex v
- When all edges of v have been explored, backtrack to explore edges leaving the vertex from which v was discovered (its *predecessor*)
- "Search as deep as possible first"
- Whenever a vertex v is discovered during a scan of the adjacency list of an already discovered vertex u, DFS records this event by setting predecessor $\pi[v]$ to u.

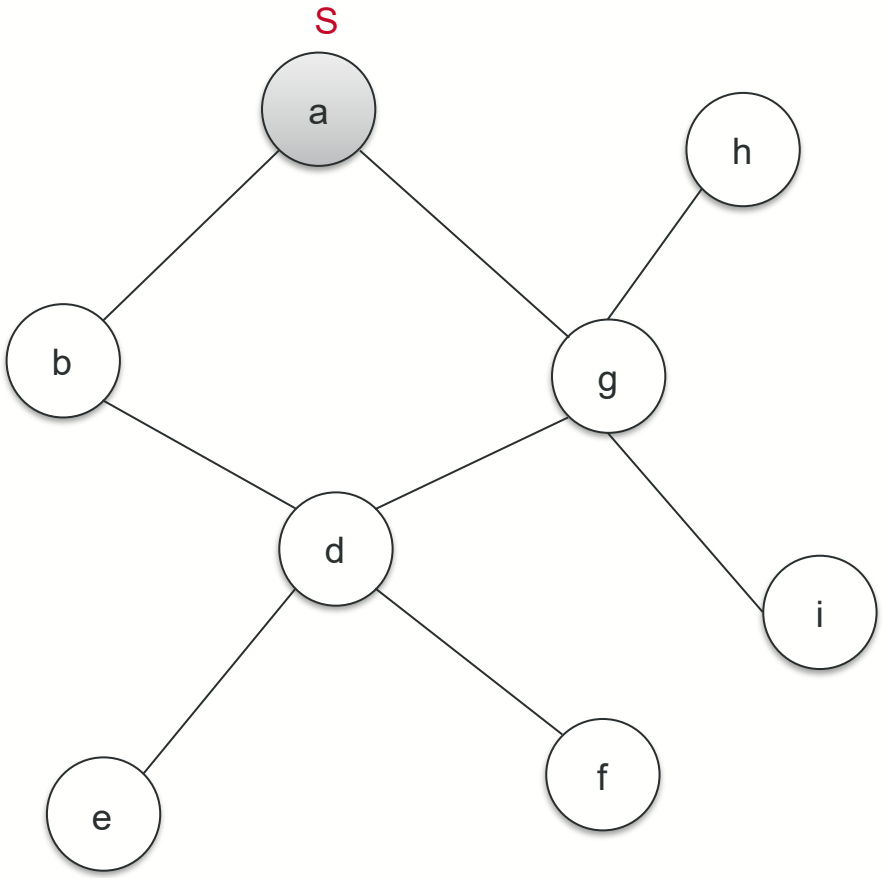
DFS example



Discovered:

Explored (adjacent nodes are expanded):

DFS example

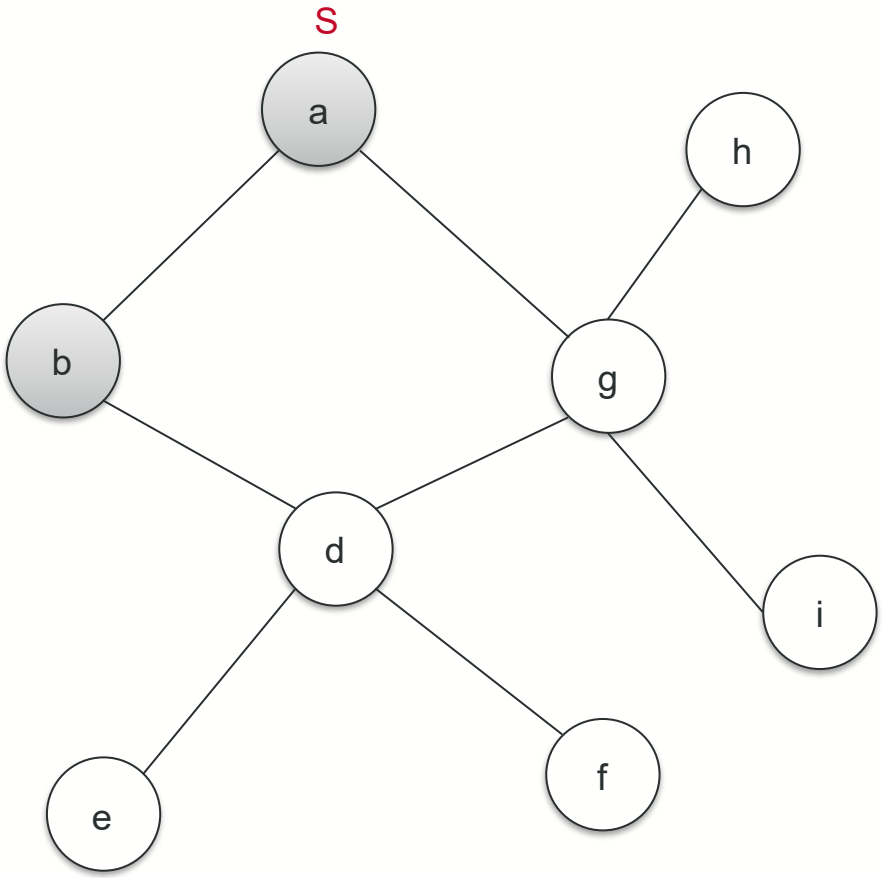


Discovered:

a									

Explored (adjacent nodes are expanded):

DFS example

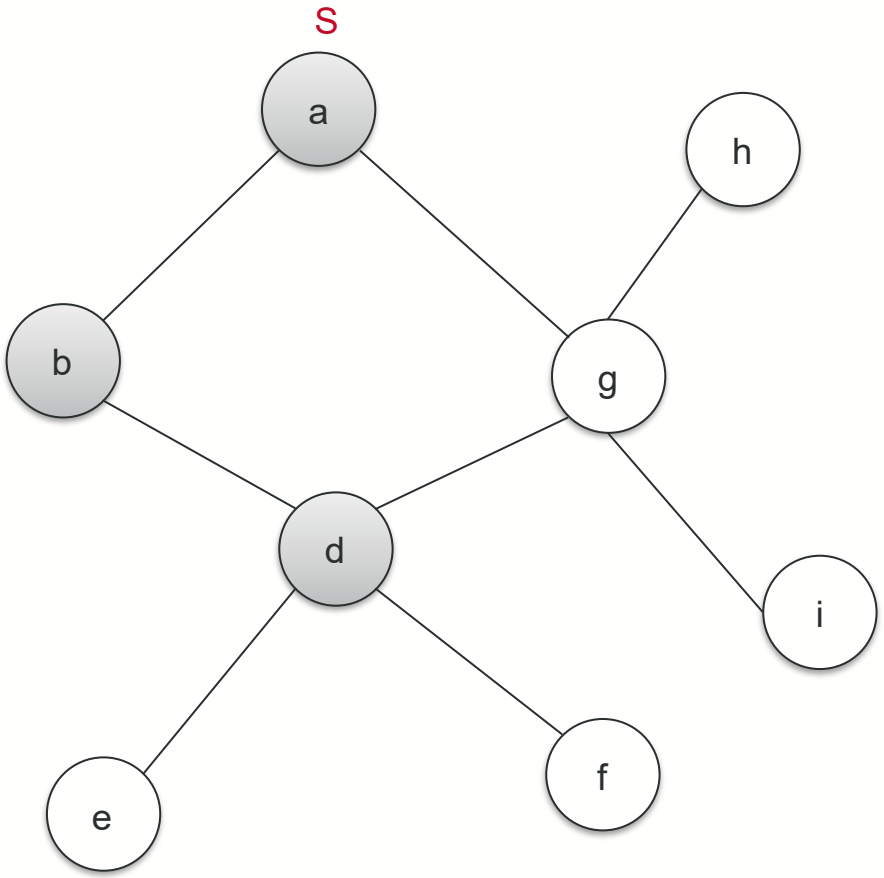


Discovered:

a	b								

Explored (adjacent nodes are expanded):

DFS example

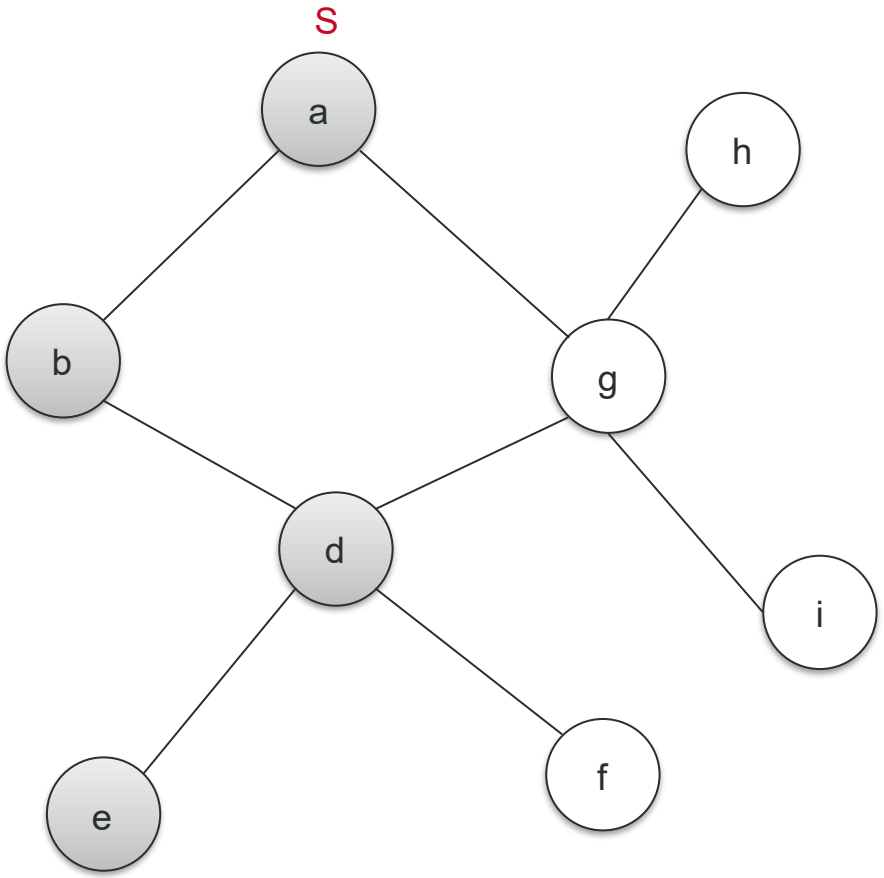


Discovered:

a	b	d							

Explored (adjacent nodes are expanded):

DFS example

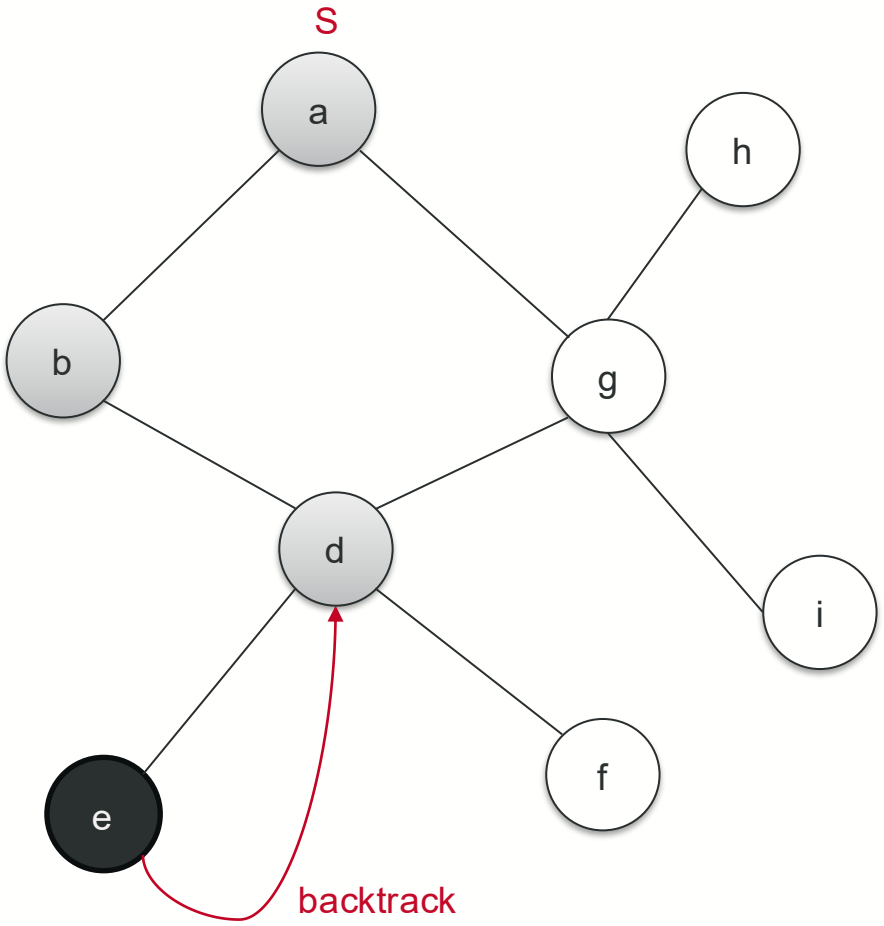


Discovered:

a	b	d	e						

Explored (adjacent nodes are expanded):

DFS example



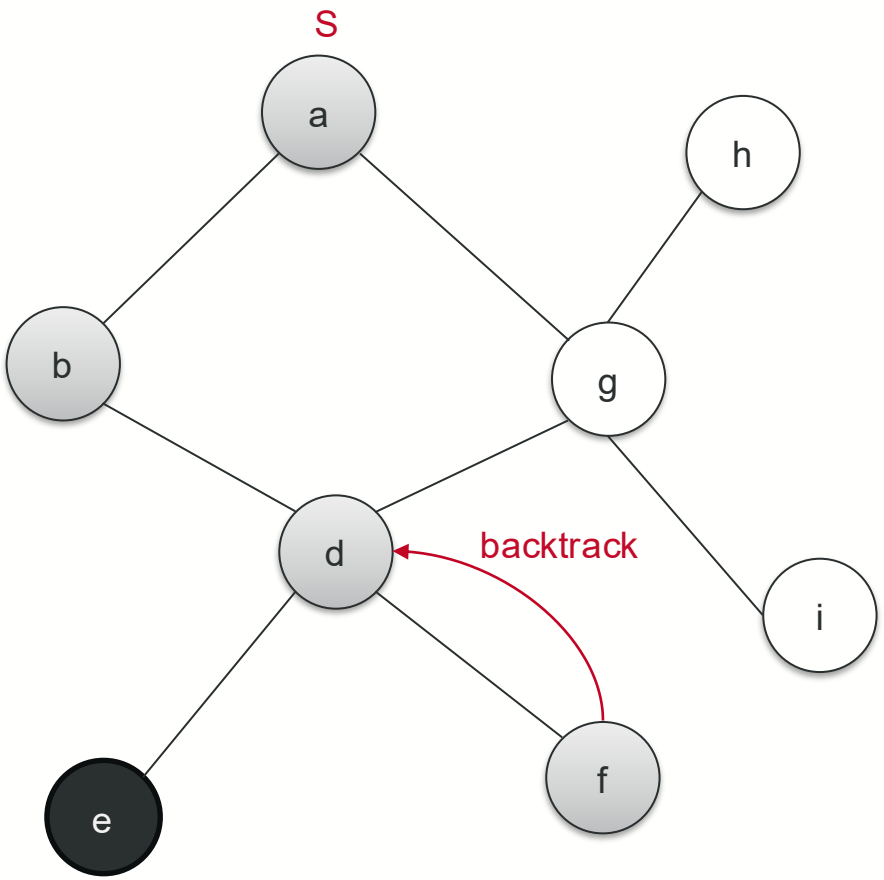
Discovered:

a	b	d	e						

Explored (adjacent nodes are expanded):

e									

DFS example



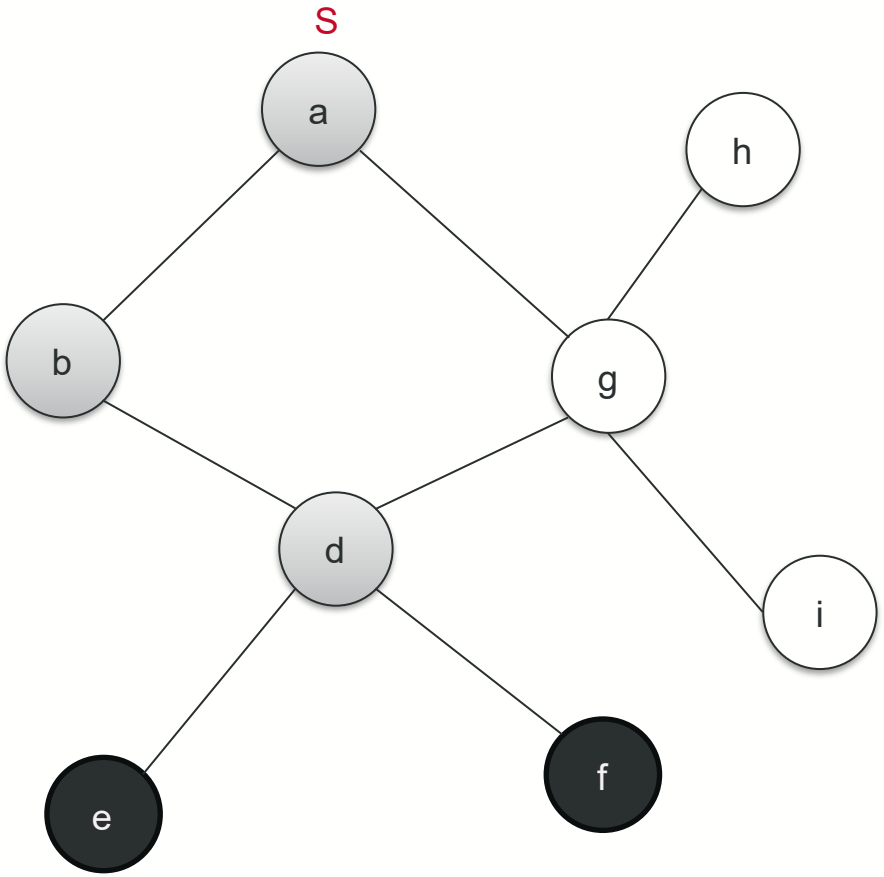
Discovered:

a	b	d	f						

Explored (adjacent nodes are expanded):

e									

DFS example



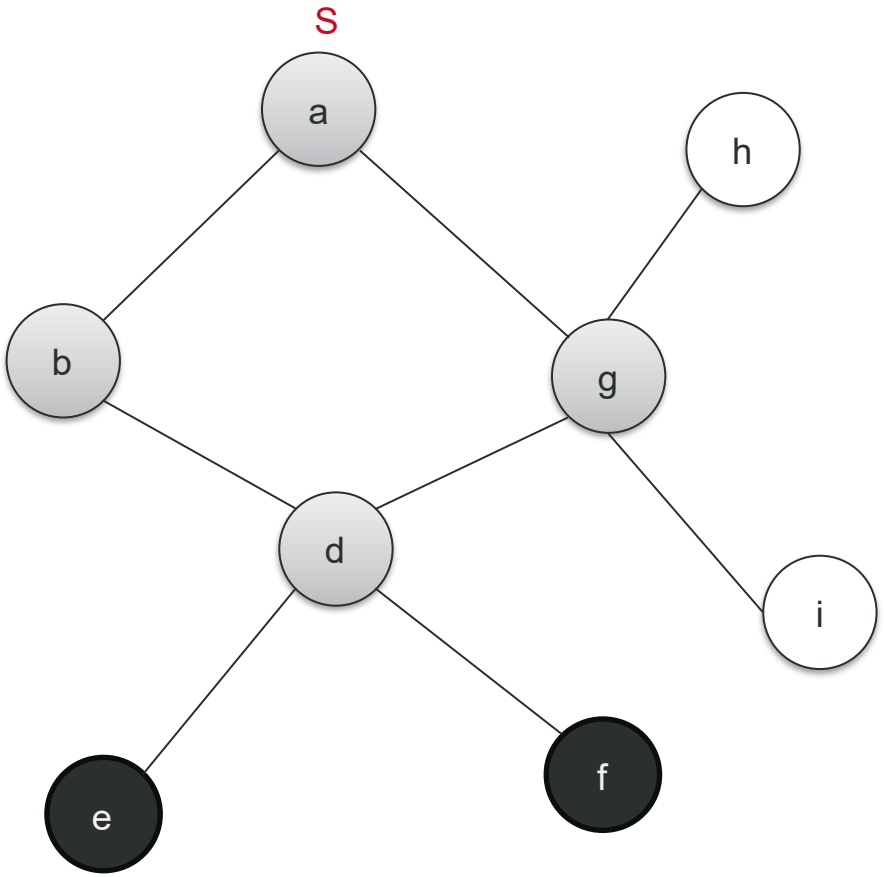
Discovered:

a	b	d	f						

Explored (adjacent nodes are expanded):

e	f								

DFS example



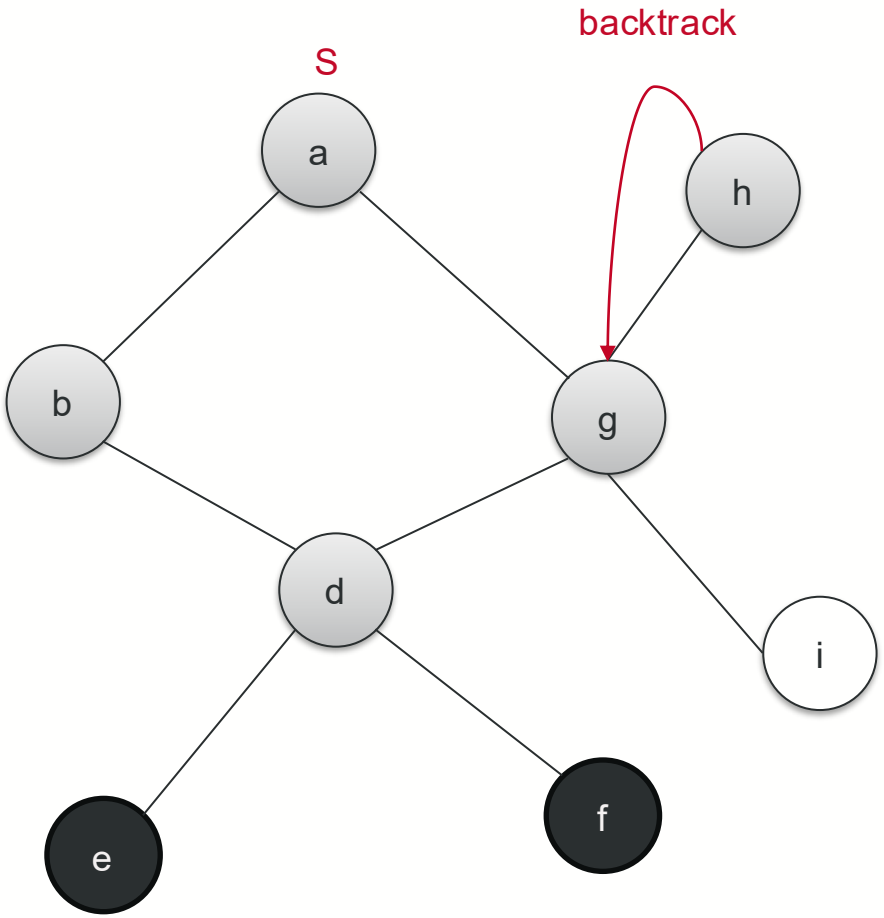
Discovered:

a	b	d	g						

Explored (adjacent nodes are expanded):

e	f								

DFS example



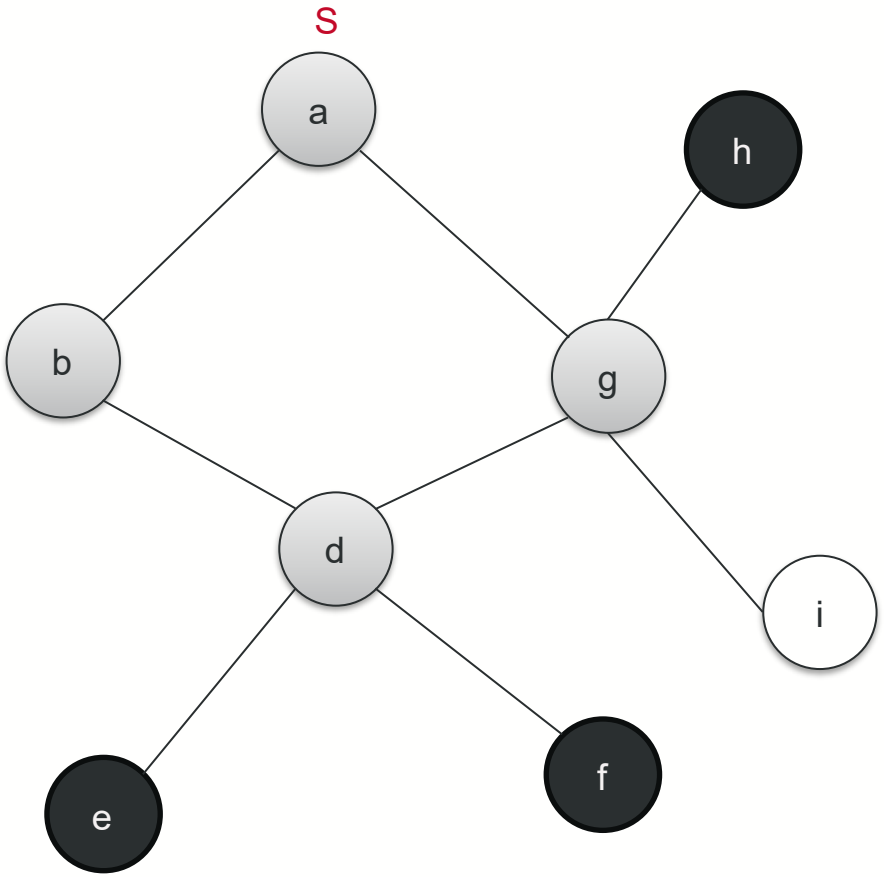
Discovered:

a	b	d	g	h					

Explored (adjacent nodes are expanded):

e	f								

DFS example



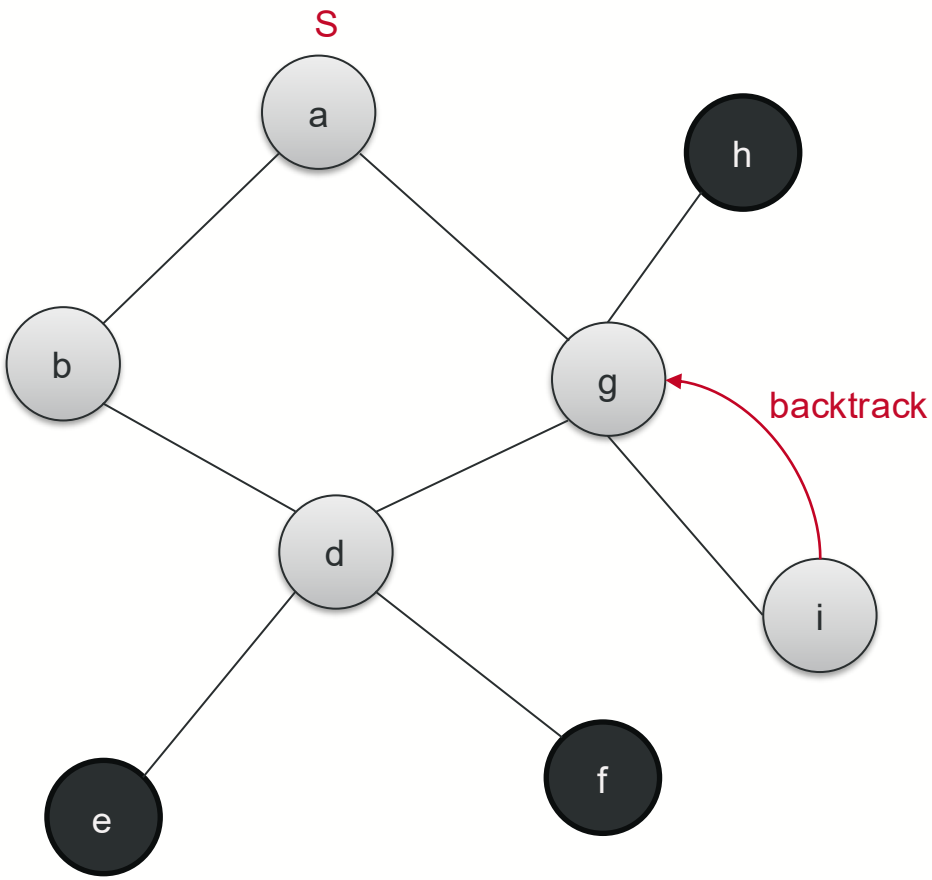
Discovered:

a	b	d	g	h					

Explored (adjacent nodes are expanded):

e	f	h							

DFS example



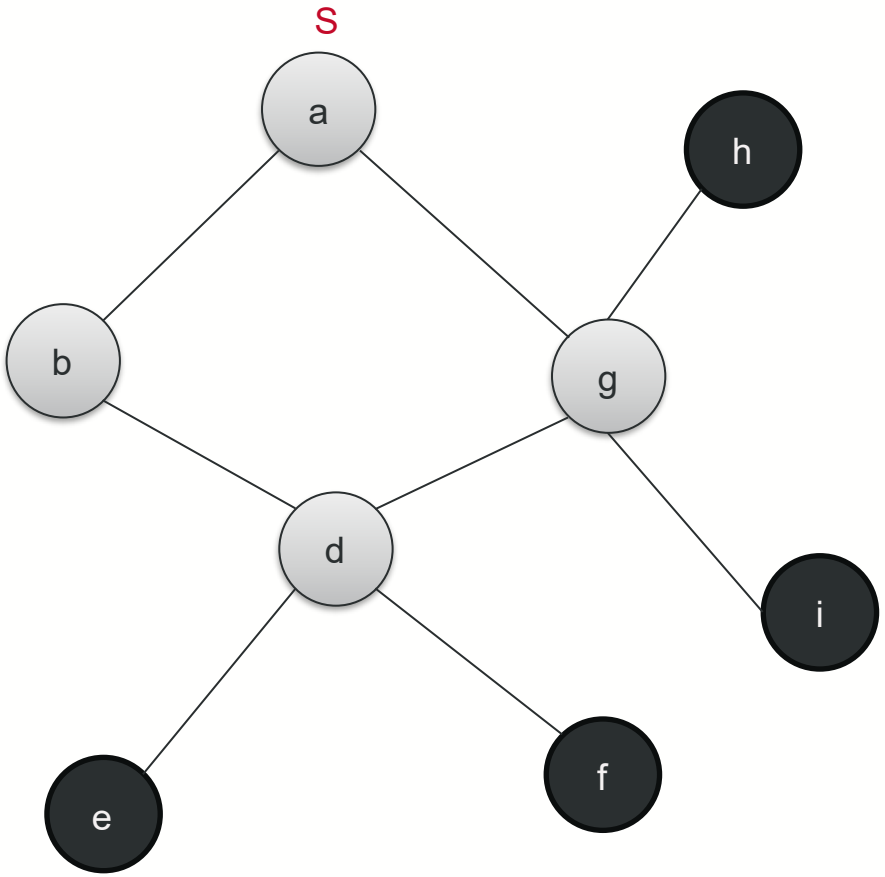
Discovered:

a	b	d	g	i					

Explored (adjacent nodes are expanded):

e	f	h							

DFS example



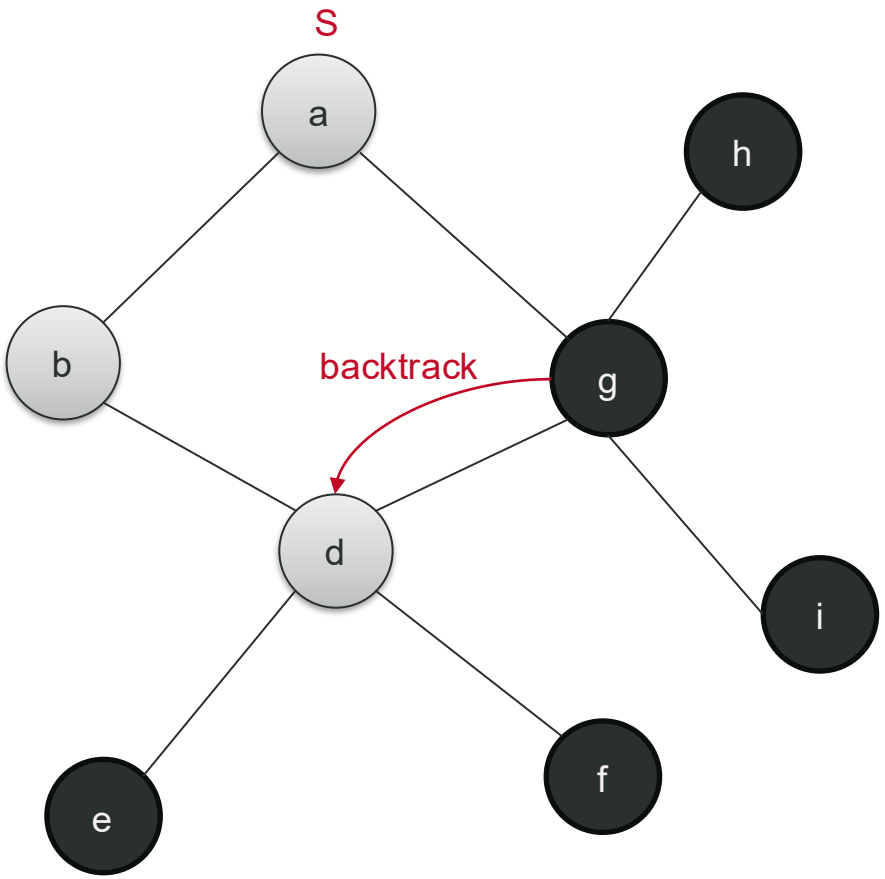
Discovered:

a	b	d	g	i					

Explored (adjacent nodes are expanded):

e	f	h	i						

DFS example



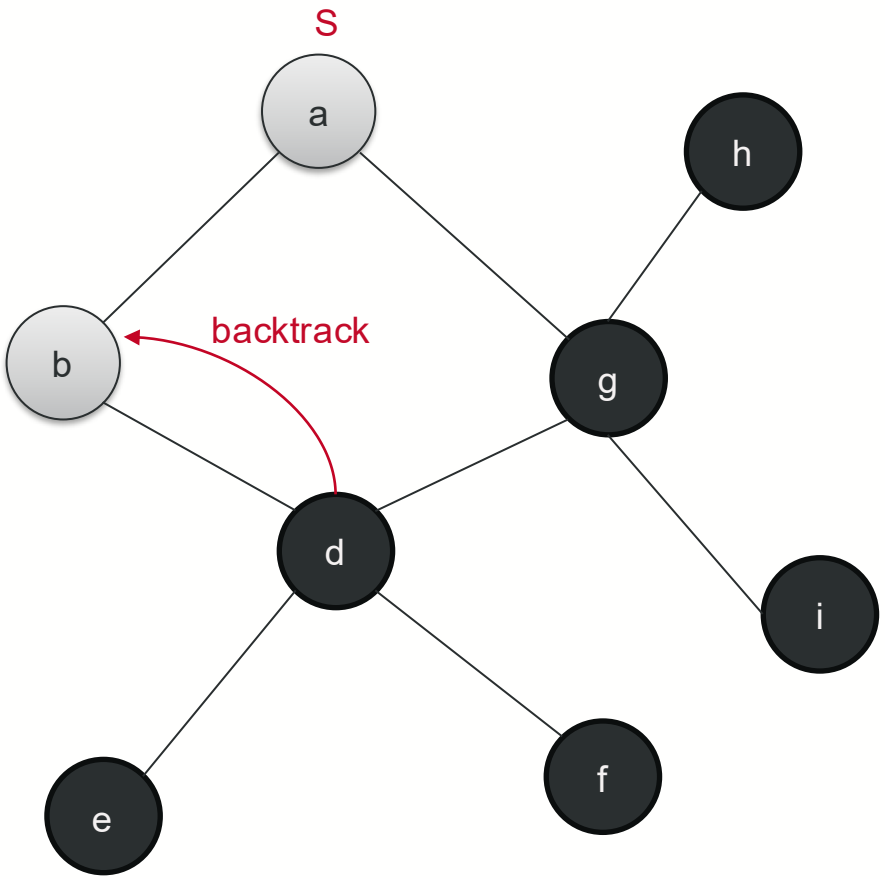
Discovered:

a	b	d	g						

Explored (adjacent nodes are expanded):

e	f	h	i	g					

DFS example



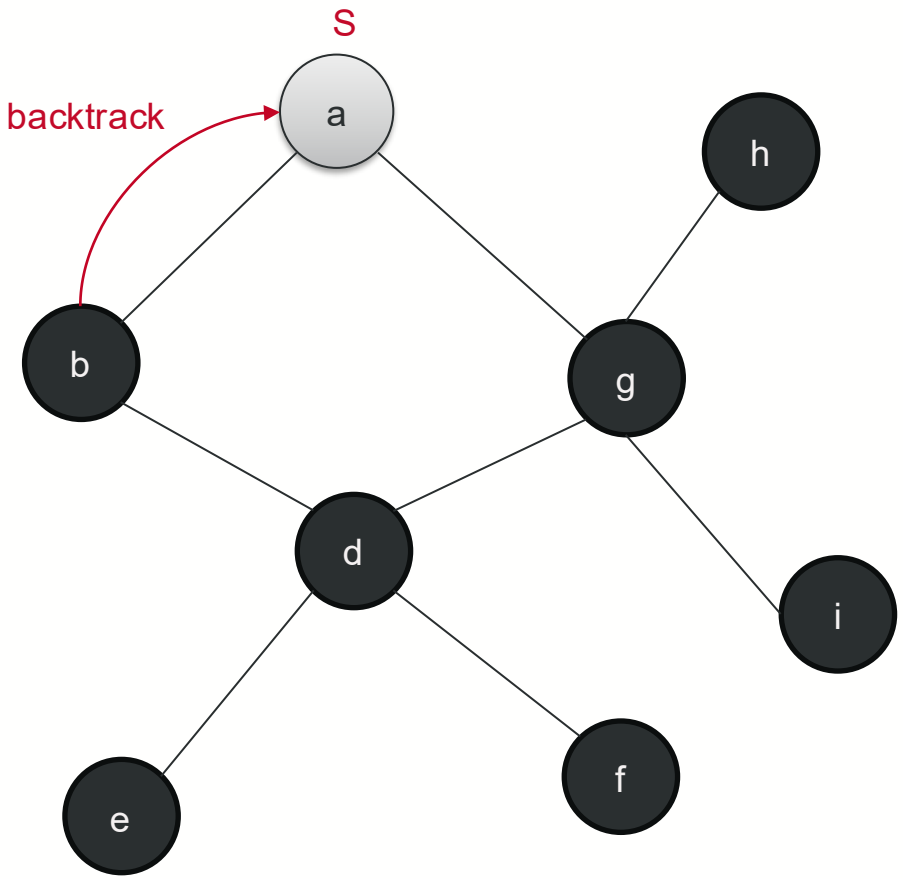
Discovered:

a	b	d							

Explored (adjacent nodes are expanded):

e	f	h	i	g	d				

DFS example



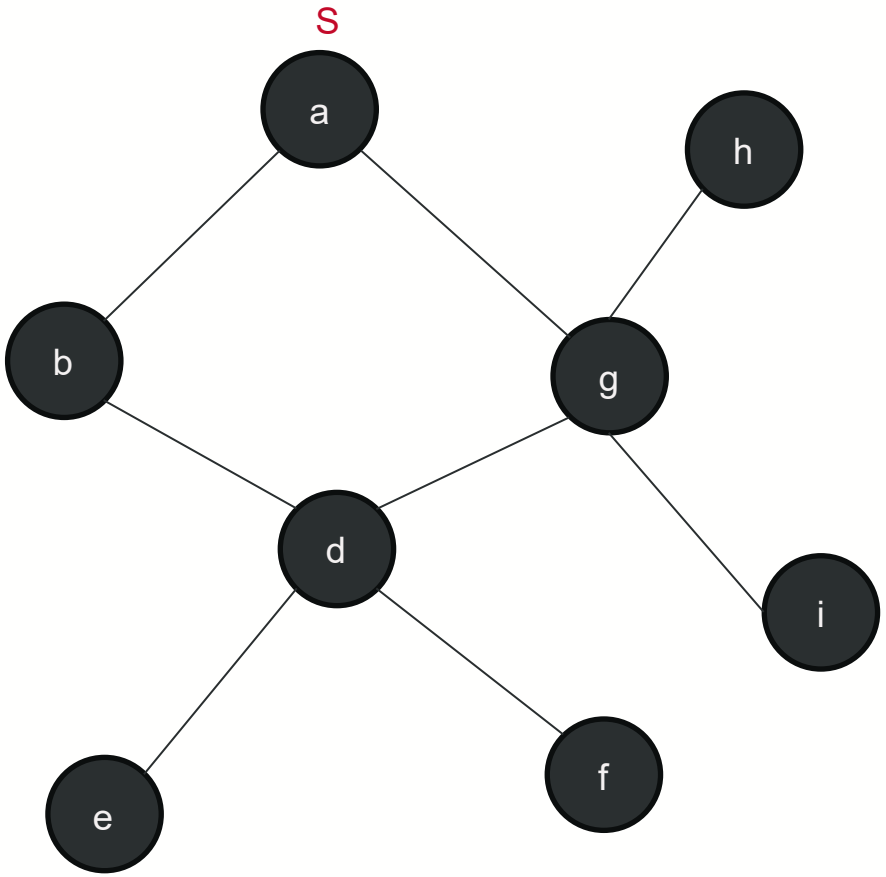
Discovered:

a	b								

Explored (adjacent nodes are expanded):

e	f	h	i	g	d	b			

DFS example



Discovered:

a									

Explored (adjacent nodes are expanded):

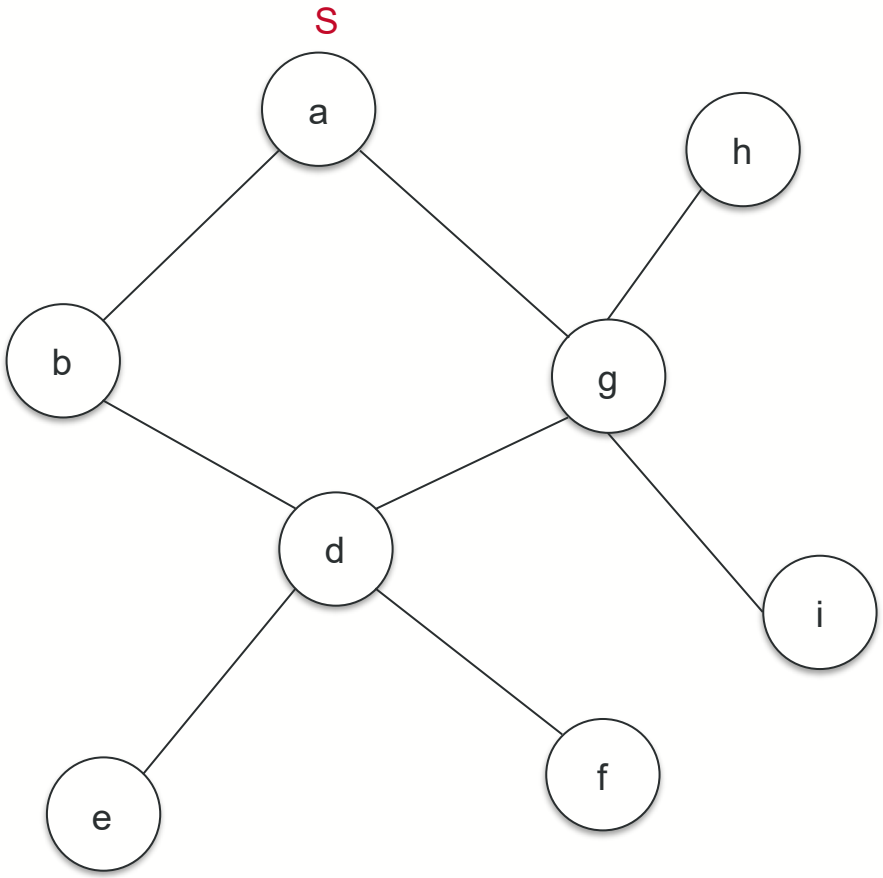
e	f	h	i	g	d	b	a		

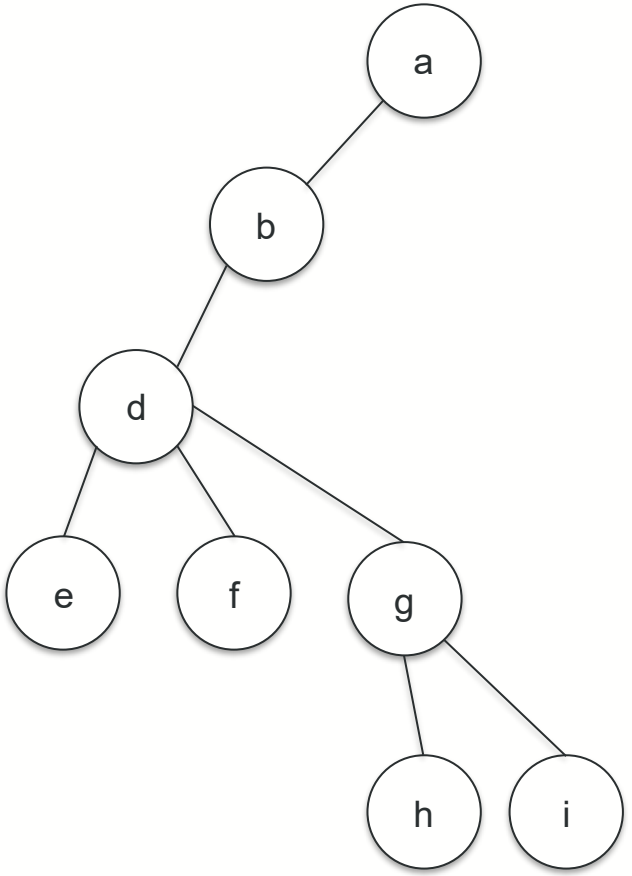
Depth-First Trees

- Coloring scheme is the same as BFS. The predecessor subgraph of DFS is $G_\pi = (V, E_\pi)$ where $E_\pi = \{(\pi[v], v) : v \in V \text{ and } \pi[v] \neq NIL\}$. The predecessor subgraph G_π forms a depth-first forest composed of several depth-first trees. The edges in E_π are called tree edges.

- Each vertex u has 2 timestamps: d[u] records when u is first discovered (grayed) and f[u] records when the search finishes (blackens). For every vertex u, d[u] < f[u].

Depth-First Tree

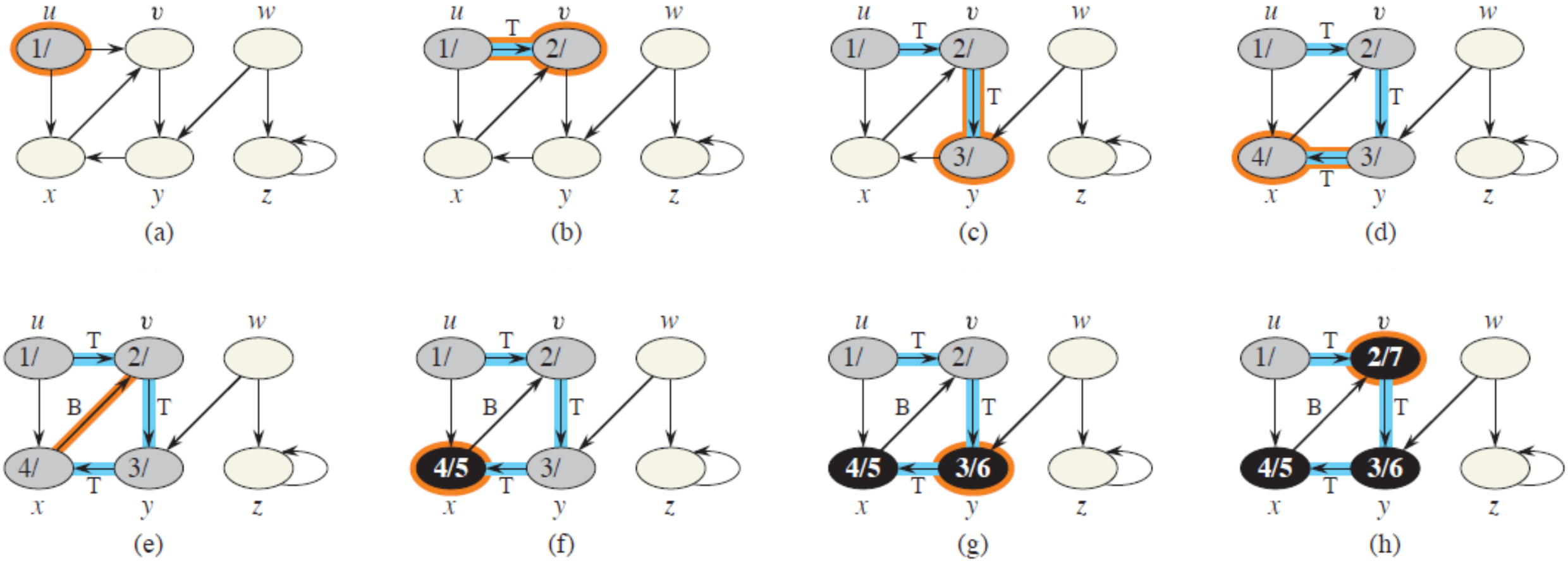




* Try to find other valid depth-first search trees

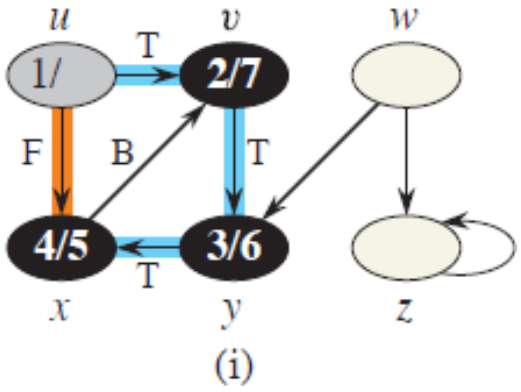
Operations of DFS

T: Tree edges
F: Forward edges
B: Back edges
C: Cross edges

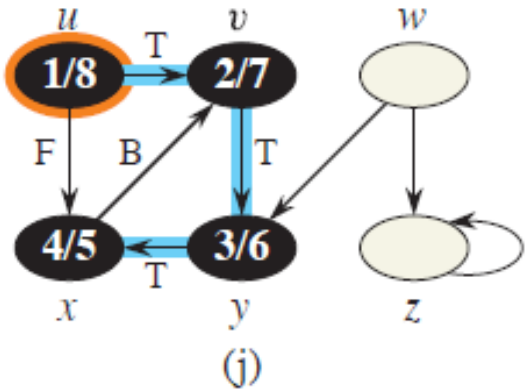


Operations of DFS

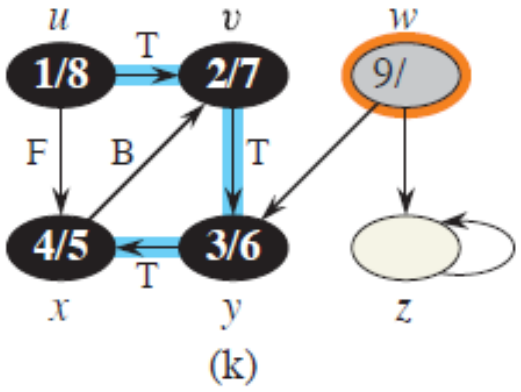
T: Tree edges
F: Forward edges
B: Back edges
C: Cross edges



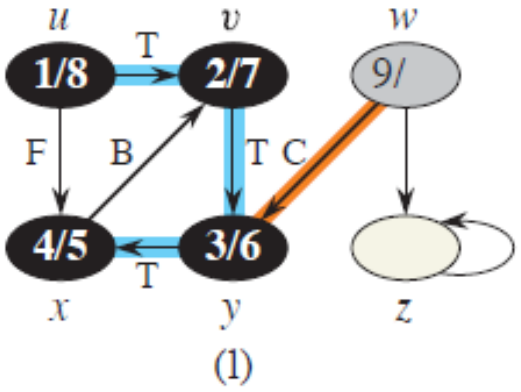
(i)



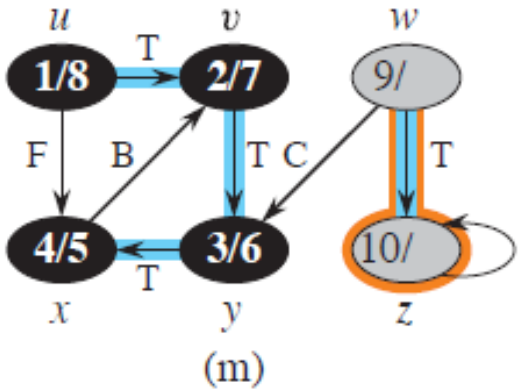
(j)



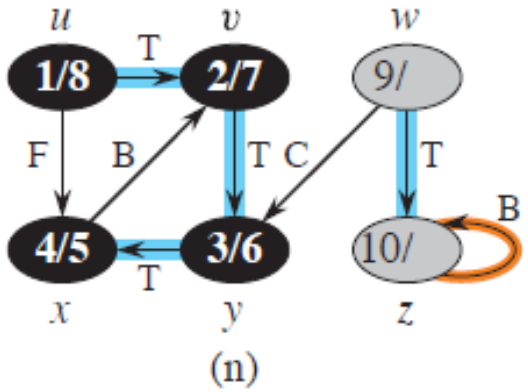
(k)



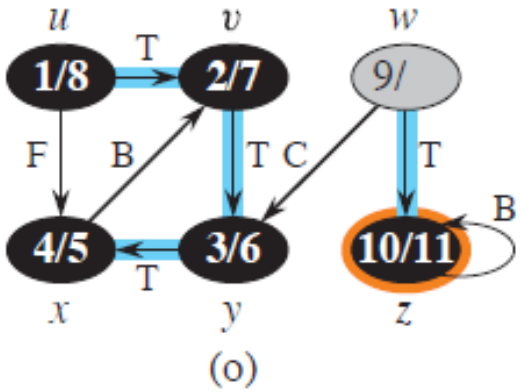
(l)



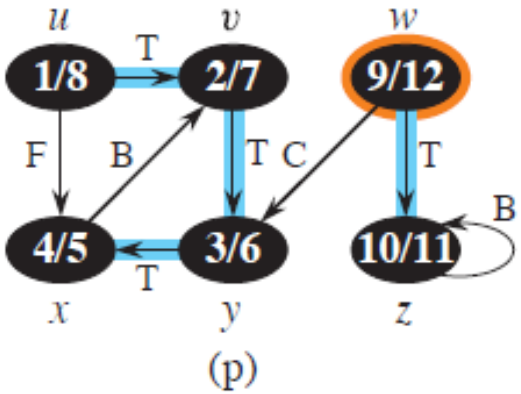
(m)



(n)



(o)



(p)

DFS(G)

1. for each vertex $u \in V[G]$
2. do $color[u] \leftarrow \text{WHITE}$
3. $\pi[u] \leftarrow \text{NIL}$
4. $time \leftarrow 0$
5. for each vertex $u \in V[G]$
6. do if $color[v] = \text{WHITE}$
7. then DFS-Visit(v)

DFS-Visit(u)

1. $color[u] \leftarrow \text{GRAY}$
   $\triangledown$ White vertex u has been discovered
2. $d[u] \leftarrow ++time$
3. for each vertex $v \in Adj[u]$
4. do if $color[v] = \text{WHITE}$
5. then $\pi[v] \leftarrow u$
6. DFS-Visit(v)
7. $color[u] \leftarrow \text{BLACK}$
   $\triangledown$ Blacken u; it is finished.
8. $f[u] \leftarrow time++$

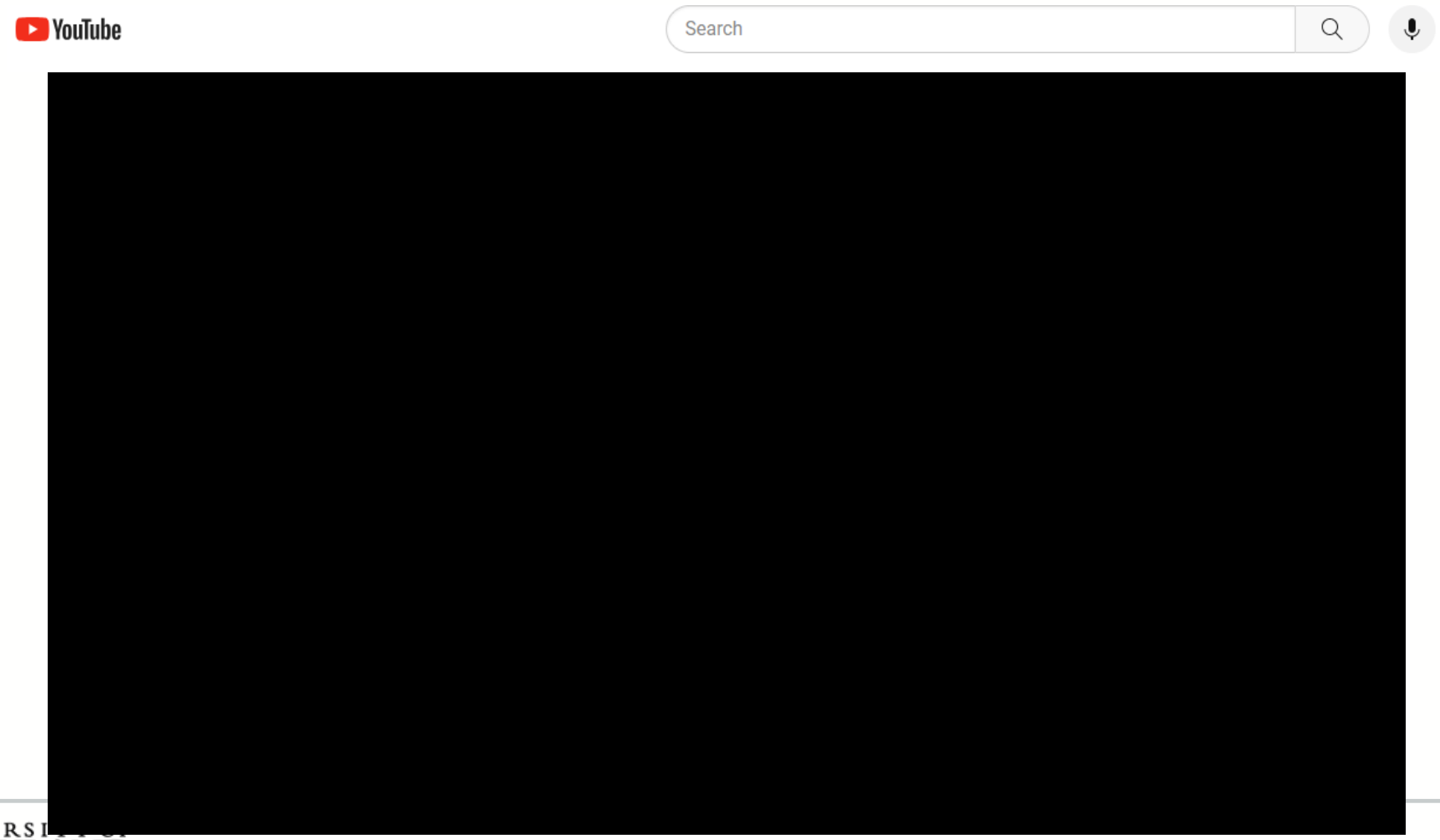
Analysis of DFS

- Loops on lines 1-2 & 5-7 take $\Theta(V)$ time, excluding time to execute DFS-Visit.

- DFS-Visit is called once for each white vertex $v \in V$ when it's painted gray the first time. Lines 3-6 of DFS-Visit is executed $|\text{Adj}[v]|$ times. The total cost of executing DFS-Visit is $\sum_{v \in V} |\text{Adj}[v]| = \Theta(E)$

- Total running time of DFS is $\Theta(V+E)$.

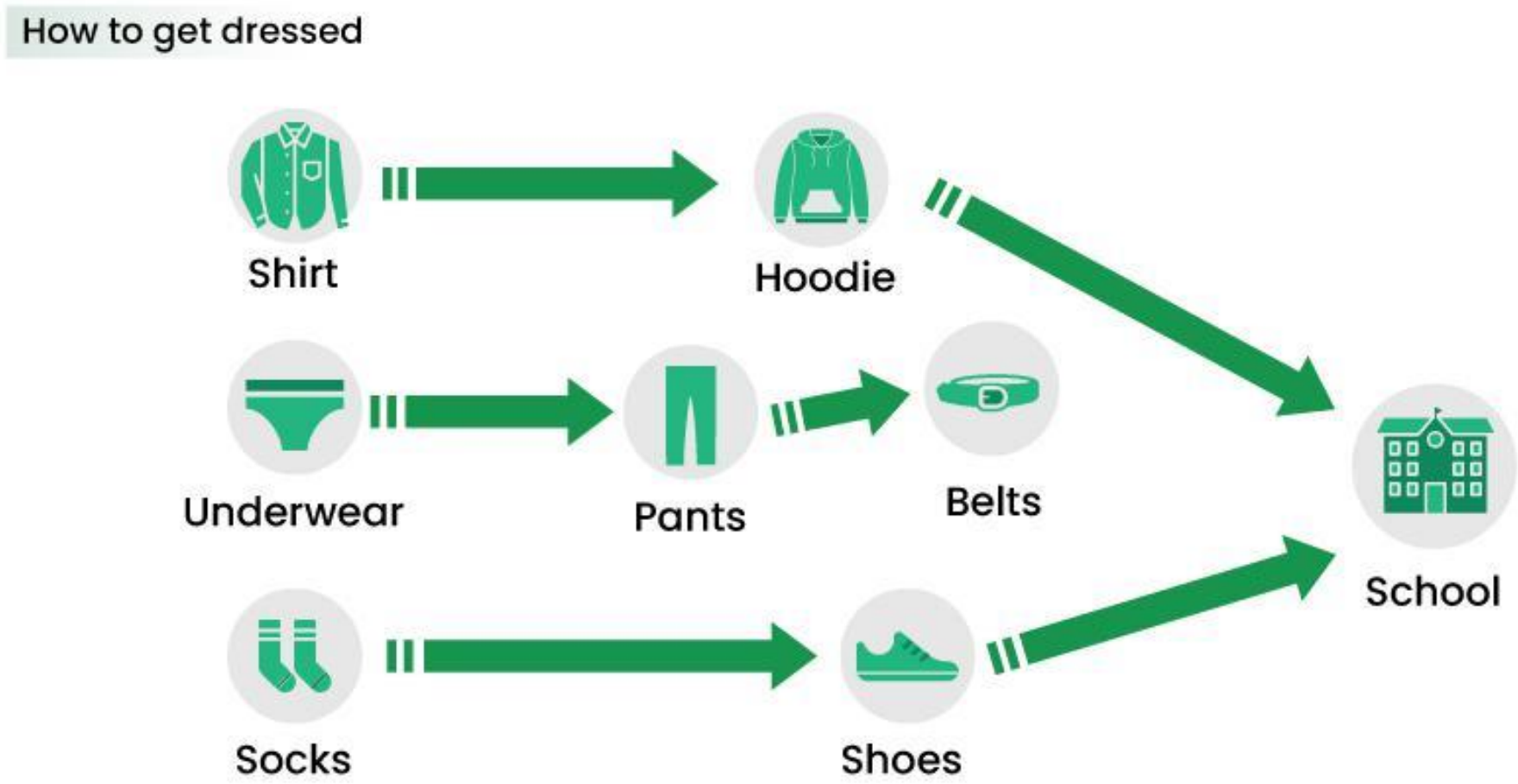
DFS vs. BFS



What DFS Offers

- Cycle Detection
- Topological Sorting
- Connected Components
- Path Finding
  - Straightforward implementation for finding any path from a source to a destination
- Generally uses less memory than BFS.
  - Only store the nodes on the current path

Topological Sorting Example



Multiple Topological ordering for a graph

Image: geeksforgeeks.org

Topological Sorting Example

Image: geeksforgeeks.org